

Erstellen einer JTL Wawi Cloud App in Form eines BI-Tools

Bericht zur betrieblichen Projektarbeit von

Felix Bock

zur Erlangung des Abschlusses

als **Fachinformatiker**

Fachrichtung

Anwendungsentwicklung

Ausbildungsbetrieb: RIS Web- & Software-Development GmbH & Co. KG

Projektbetreuer: [?????????]

Prüflingsnummer: (166)-95084

Ausführungszeit: 04.04.2026 - 22.05.2026

Inhaltsverzeichnis

Abbildungsverzeichnis	II
Tabellenverzeichnis	III
Quelltextverzeichnis	III
Abkürzungsverzeichnis	IV
1 Einleitung	1
1.1 Projektbeschreibung	1
1.2 Projektziel	1
1.3 Projektumfeld	2
1.4 Projektbegründung	2
1.5 Projektschnittstellen	2
1.6 Projektabgrenzung	2
2 Projektplanung	3
2.1 Projektphasen	3
2.2 Ressourcenplanung	3
2.3 Entwicklungsprozess	3
2.4 Iterationsplanung	3
3 Analysephase	5
3.1 Ist-Analyse	5
3.2 Wirtschaftlichkeitsanalyse	5
3.2.1 Make-or-Buy-Entscheidung	5
3.2.2 Projektkosten	5
3.2.3 Amortisationsdauer	5
3.3 Nicht-monetäre Vorteile	6
3.4 Anwendungsfälle	6
3.5 Lastenheft / Fachkonzept	6
4 Entwurfsphase	7
4.1 Zielplattform	7
4.2 Architekturdesign	8
4.3 Entwurf der Benutzeroberfläche	10
4.4 Datenmodell und API-Kommunikation	10
4.4.1 Selbst implementierte Backend-Endpunkte	10
4.4.2 JTL AppBridge-Kommunikation	11
4.4.3 OAuth2-Authentifizierungskonzept	12
4.5 Pflichtenheft	12
5 Implementierungsphase	13
5.1 Implementierung der Projektstruktur	13
5.2 Implementierung der Geschäftslogik	13
5.2.1 Session-Token-Verifikation	13
5.2.2 ERP-API-Proxy	13
5.2.3 GraphQL-Datenabruf	14
5.3 Implementierung der Benutzeroberfläche	15
5.3.1 Setup-Seite mit Token-Verifikation	15
5.3.2 Pane-Widget mit Event-Subscription	15
5.3.3 ERP-Dashboard mit Kennzahlen	16

6	Abnahme- und Einführungsphase	17
6.1	Qualitätssicherung	17
6.2	Abnahme durch den Auftraggeber	17
6.3	Deployment und Einführung	18
7	Dokumentation	19
8	Fazit	20
8.1	Soll-/Ist-Vergleich	20
8.2	Lessons Learned	20
8.3	Ausblick	21
A	Anhang	22
A.1	Detaillierte Zeitplanung	22
A.2	Verwendete Ressourcen	22
A.3	Lastenheft	24
A.4	Mockups der Benutzeroberfläche	26
A.5	Pflichtenheft	26
A.6	Iterationsplan	29
A.7	Quellcode-Auszüge	29
A.7.1	Backend: OAuth2-Token-Anfrage	29
A.7.2	Backend: Session-Token-Verifikation	30
A.7.3	Backend: ERP-API-Proxy-Route	30
A.7.4	Backend: Tenant-Mapping	31
A.7.5	Frontend: App-Routing	31
A.7.6	Frontend: Setup-Seite	31
A.7.7	Frontend: Pane-Widget	32
A.7.8	Frontend: ERP-Dashboard-Komponente	32
A.7.9	Frontend: React-Einstiegspunkt	33
A.8	Screenshot der Anwendung	34
A.9	Entwicklerdokumentation	34
A.10	Benutzerhandbuch	38
A.11	Eigenständigkeitserklärung	40

Abbildungsverzeichnis

4.1	Systemarchitektur, Übersicht	9
4.2	Komponentendiagramm der Systemarchitektur	10
4.3	Sequenzdiagramm: Authentifizierungs- und Token-Verifikationsablauf	12
A.1	Mockup der Dashboard-Hauptansicht	26
A.2	Screenshot des fertigen ERP-Dashboards	34
A.3	Dashboard-Ansicht (Screenshot)	39

Tabellenverzeichnis

2.1	Grobe Zeitplanung	3
3.1	Kostenaufstellung	5
4.1	Nutzwertanalyse zur Auswahl des Backend-Frameworks	8
4.2	Selbst implementierte Backend-Application Programming Interface (API)-Endpunkte	11
4.3	GraphQL-Operationen und Rückgabetypen	11

4.4	AppBridge-Methoden und Events	11
6.1	Testfälle Qualitätssicherung	17
8.1	Soll-/Ist-Vergleich	20
A.1	Funktionale Anforderungen (Lastenheft)	24
A.2	Nicht-funktionale Anforderungen (Lastenheft)	25
A.3	Anforderungs-Umsetzungs-Matrix (Pflichtenheft)	27
A.4	Auftragsstatus im Dashboard	40
A.5	Häufige Probleme und Lösungen	40

Listings

5.1	Projektstruktur	13
5.2	Paralleler GraphQL-Datenabruf (api.ts, Auszug)	14
A.1	OAuth2-Token-Anfrage mit Basic Auth (index.ts)	29
A.2	Session-Token-Verifikation mit JWKS (index.ts)	30
A.3	Generischer REST-Proxy-Endpunkt (index.ts)	30
A.4	Tenant-Mapping Endpunkt (index.ts)	31
A.5	App-Komponente mit URL-basiertem Routing (App.tsx)	31
A.6	Setup-Seite mit AppBridge-Token-Flow (SetupPage.tsx)	31
A.7	Pane-Widget mit Event-Subscription (PanePage.tsx)	32
A.8	ERP-Dashboard: Datenabruf und Darstellung (ErpPage.tsx, Auszug)	33
A.9	main.tsx mit AppBridge-Initialisierung	33
A.10	Projekt einrichten	34
A.11	Umgebungsvariablen (packages/backend/.env)	34
A.12	Optionale Frontend-Variable	35
A.13	Entwicklungsserver starten	35
A.14	POST /connect-tenant - Beispiel-Request	35
A.15	POST /connect-tenant - Erfolgs-Response	35
A.16	POST /graphql - Beispiel-Request	35
A.17	JSDoc-Dokumentation: fetchDashboard()	36
A.18	JSDoc-Dokumentation: computeDateRange()	36
A.19	JSDoc-Dokumentation: Hilfsfunktionen	37
A.20	TypeScript-Interfaces (api.ts)	37
A.21	Test-Script starten (packages/frontend)	38

Abkürzungsverzeichnis

API Application Programming Interface.

BI Business Intelligence.

CORS Cross-Origin Resource Sharing.

EdDSA Edwards-curve Digital Signature Algorithm.

HTTPS Hypertext Transfer Protocol Secure.

IHK Industrie- und Handelskammer.

IPC Inter-Process Communication.

ISO International Organization for Standardization.

JTL JTL-Software GmbH.

JWKS JSON Web Key Set.

JWT JSON Web Token.

KPI Key Performance Indicator.

LTS Long-Term Support.

REST Representational State Transfer.

RIS RIS Web- & Software-Development GmbH & Co. KG.

SPA Single Page Application.

UI User Interface.

URL Uniform Resource Locator.

1 Einleitung

Die Projektdokumentation schildert den Ablauf des IHK-Abschlussprojektes, welches im Rahmen der Ausbildung zum Fachinformatiker Anwendungsentwicklung durchgeführt wurde. Ausbildungsbetrieb ist die RIS Web- & Software-Development GmbH & Co. KG (RIS) mit Sitz in Regensburg. Das Unternehmen wurde im Jahr 2015 gegründet und beschäftigt derzeit acht Mitarbeiter. Als IT-Dienstleister ist RIS auf die Entwicklung individueller Web- und Softwarelösungen spezialisiert und betreut insbesondere E-Commerce-Unternehmen in Deutschland, Österreich und der Schweiz.

1.1 Projektbeschreibung

Ein zentraler Schwerpunkt der Unternehmenstätigkeit liegt auf der Implementierung und Erweiterung der Systemlandschaft der JTL-Software. Die JTL-Software-Suite bildet typische Geschäftsprozesse im Onlinehandel ab. Kernbestandteile sind JTL-Wawi als zentrales Warenwirtschaftssystem (Artikelverwaltung, Auftragsabwicklung, Lagerverwaltung), Shop- und Marktplatzanbindungen, über die JTL-Wawi mit externen Verkaufsplattformen wie Amazon oder eBay über standardisierte Datenschnittstellen verbunden wird, sowie cloudbasierte Erweiterungen über die JTL-Cloud. Aktuell wird die JTL-Wawi schrittweise in eine browserbasierte Umgebung innerhalb der JTL-Cloud überführt. Diese befindet sich noch in der Beta-Phase und soll perspektivisch die technologische Grundlage der zukünftigen JTL-Systemlandschaft darstellen. Damit verbunden ist auch eine neue API-Struktur, die externen Anwendungen den Zugriff auf Geschäftsdaten ermöglichen soll. Viele der von RIS betreuten Kunden verfügen über umfangreiche Unternehmensdaten innerhalb der JTL-Wawi, darunter Umsatzdaten, Lagerbestände, Auftragsvolumen und weitere betriebswirtschaftlich relevante Kennzahlen. Eine flexible, mobil nutzbare und visuell aufbereitete Auswertung dieser Daten steht derzeit jedoch nicht zur Verfügung. In diesem Projekt soll eine Business Intelligence (BI)-Applikation als Prototyp entwickelt werden, die diese Lücke schließt.

1.2 Projektziel

Ziel des Projekts ist die Konzeption und Entwicklung eines Prototypen einer Business-Intelligence-Applikation zur Visualisierung ausgewählter Geschäftsdaten aus der JTL-Systemlandschaft. Im Rahmen des Projekts soll die neue API-Schnittstelle der JTL-Cloud analysiert und angebunden werden, um definierte betriebswirtschaftliche Kennzahlen, beispielsweise Umsätze, Auftragszahlen und Lagerbestände, automatisiert abzurufen. Die Daten werden strukturiert aufbereitet und in einer benutzerfreundlichen Oberfläche grafisch dargestellt. Aufgrund des Beta-Status der JTL-Cloud wird die Anwendung zunächst als Pilotprojekt umgesetzt. Ziel ist es, die technische Umsetzbarkeit, Stabilität und Integrationsfähigkeit der Schnittstelle zu bewerten. Sollte sich die Lösung als stabil und praxistauglich erweisen, besteht die Möglichkeit, diese zu einem marktfähigen Produkt im Portfolio von RIS weiterzuentwickeln.

1.3 Projektumfeld

Auftraggeber des Projekts ist die RIS. Die Ergebnisse richten sich an E-Commerce-Kunden des Unternehmens, die ihre Geschäftsdaten aus der JTL-Wawi in aufbereiteter Form abrufen möchten. Darüber hinaus hat RIS ein strategisches Interesse an einer frühen technischen Evaluierung der neuen JTL-Cloud-API, um die eigene Beratungskompetenz für cloudbasierte JTL-Erweiterungen auszubauen.

1.4 Projektbegründung

Datengetriebene Unternehmensentscheidungen werden heute in fast jeder Branche getroffen, im E-Commerce sind dafür aktuelle Kennzahlen besonders entscheidend. Ohne eine geeignete Visualisierungsmöglichkeit fehlt Entscheidern eine kompakte, ortsunabhängige Übersicht über die wirtschaftliche Situation ihres Unternehmens. Bestehende Auswertungsmöglichkeiten in der JTL-Wawi sind funktional eingeschränkt oder nicht für eine intuitive, mobile Nutzung ausgelegt. Für RIS als JTL-Dienstleister besteht zusätzlich die strategische Notwendigkeit, die neue Cloud-API frühzeitig technisch zu evaluieren und deren Praxistauglichkeit zu prüfen.

1.5 Projektschnittstellen

Das System kommuniziert über folgende Schnittstellen. Die Kommunikation mit allen externen Diensten erfolgt verschlüsselt über Hypertext Transfer Protocol Secure (HTTPS).

- **JTL-Cloud Auth-Server** (<https://auth.jtl-cloud.com>): Stellt über OAuth2, einem offenen Standard zur sicheren Zugriffsautorisierung, Zugriffstoken aus. Der verwendete `client_credentials`-Flow ist ein Server-zu-Server-Verfahren, bei dem die Anwendung sich direkt mit App-ID und App-Secret ausweist, ohne dass eine Nutzerinteraktion erforderlich ist.
- **JTL-Cloud API** (<https://api.jtl-cloud.com>): Representational State Transfer (REST)-API, die einen standardisierten Datenaustausch über das HTTP-Protokoll ermöglicht, zum Abruf von Geschäftsdaten wie Aufträgen, Artikeln und Lagerbeständen
- **Frontend** (React-Single Page Application (SPA)): Kommuniziert mit dem eigenen Backend über eine interne REST-API

1.6 Projektabgrenzung

Da der Projektumfang auf 80 Stunden begrenzt ist, sind folgende Punkte ausdrücklich **nicht** Bestandteil des Abschlussprojekts:

- Entwicklung eines produktionsreifen, marktfähigen Produkts (ausschließlich Prototyp)
- Anbindung weiterer Systeme außerhalb der JTL-Cloud-API
- Mandantenfähigkeit für mehrere gleichzeitige JTL-Kunden
- Migration oder Veränderung bestehender JTL-Wawi-Daten

2 Projektplanung

2.1 Projektphasen

Für die Umsetzung des Projekts standen 80 Stunden zur Verfügung. Diese wurden vor Projektbeginn auf fünf Hauptphasen verteilt. Eine grobe Übersicht zeigt Tabelle 2.1. Die detaillierte Zeitplanung findet sich im Anhang A.1.

Tabelle 2.1: Grobe Zeitplanung

Projektphase	Geplante Zeit
Analysephase	10 h
Entwurfsphase	12 h
Implementierungsphase	51 h
Dokumentation	5 h
Abschluss	2 h
Gesamt	80 h

2.2 Ressourcenplanung

Eine Übersicht der eingesetzten Ressourcen befindet sich im Anhang A.2. Bei der Auswahl der verwendeten Software wurde ausschließlich auf kostenlose Werkzeuge gesetzt, um anfallende Projektkosten möglichst gering zu halten.

2.3 Entwicklungsprozess

Bevor mit der Realisierung des Projekts begonnen werden konnte, musste ein geeigneter Entwicklungsprozess gewählt werden. Für das Abschlussprojekt wurde ein iterativ-inkrementeller Ansatz in Anlehnung an agile Vorgehensmodelle gewählt. Die einzelnen Implementierungsphasen (Backend, Frontend, Qualitätssicherung) werden nacheinander, aber mit regelmäßiger Überprüfung der Zwischenergebnisse, durchlaufen.

2.4 Iterationsplanung

Auf Basis des gewählten Entwicklungsprozesses wurde vor Beginn der Implementierung ein Iterationsplan erstellt, der die Umsetzung in folgende Schritte gliedert:

- Einrichtung der Entwicklungsumgebung und Projektstruktur
- Implementierung der OAuth2-Authentifizierung
- Implementierung der JTL-Cloud-API-Anbindung (Datenabfragen)
- Aufbereitung und Bereitstellung der Daten über die interne API
- Implementierung der React-Frontend-Komponenten

- Implementierung von Filtern und Benutzerinteraktionen
- Qualitätssicherung und Code Review

Der vollständige Iterationsplan mit Zeitangaben befindet sich im Anhang [A.6](#).

3 Analysephase

3.1 Ist-Analyse

Wie in Abschnitt 1.1 (Projektbeschreibung) erläutert, fehlt den Kunden von RIS derzeit eine mobile, visuell aufbereitete Auswertungsmöglichkeit für ihre JTL-Wawi-Daten. Die vorhandenen integrierten Analyse-Werkzeuge der JTL-Software sind entweder auf Desktop-Nutzung beschränkt oder bieten keine flexible Filtermöglichkeit für individuelle Zeiträume und Key Performance Indicators (KPIs), also messbare Kennzahlen wie Umsatz oder Auftragsanzahl, anhand derer der Geschäftserfolg bewertet wird.

3.2 Wirtschaftlichkeitsanalyse

3.2.1 Make-or-Buy-Entscheidung

Da die Anwendung auf die JTL-Cloud-API in ihrer Beta-Phase aufbaut, war eine Kaufentscheidung strukturell ausgeschlossen: Zum Zeitpunkt der Projektumsetzung existierte kein Drittanbieterprodukt, das diese Schnittstelle unterstützt. Eine „Buy“-Option stand damit nicht zur Verfügung. Die Eigenentwicklung bietet zudem den strategischen Vorteil, dass RIS frühzeitig Erfahrungswissen mit der neuen JTL-Cloud-Plattform aufbaut, bevor diese allgemeine Marktreife erlangt, und sich damit als früher Anbieter von JTL-Cloud-Erweiterungen positioniert.

3.2.2 Projektkosten

Den Berechnungen liegt ein interner Verrechnungssatz von 15 €/h für den Auszubildenden sowie 25 €/h für Mitarbeiter zugrunde, entsprechend den betrieblichen Kostensätzen der RIS.

Tabelle 3.1: Kostenaufstellung

Vorgang	Mitarbeiter	Zeit	Gesamt
Entwicklungskosten	1 × Auszubildender	80 h	1.200,00 €
Fachgespräche	1 × Mitarbeiter	4 h	100,00 €
Code Review	1 × Mitarbeiter	4 h	100,00 €
Abnahme/Vorstellung	1 × Mitarbeiter	2 h	50,00 €
Projektkosten gesamt			1.450,00 €

3.2.3 Amortisationsdauer

Da RIS die Anwendung als Produkt an JTL-Kunden vermarkten möchte, ergibt sich die Amortisation nicht aus der Zeitersparnis beim Kunden, sondern aus den zu erwartenden Lizezeinnahmen. Als Planungsgrundlage wird folgendes Szenario zugrunde gelegt: Bei einer monatlichen Lizenzgebühr von 29 € pro Mandant und fünf zahlenden Kunden beläuft sich der monatliche Deckungsbeitrag auf 145 €. Bei Projektkosten von 1.450 € ergibt sich eine rechnerische Amortisationsdauer von zehn Monaten. Dieses Szenario stellt eine konservative Schätzung dar; bei höherer Kundenzahl oder

einem höheren Lizenzpreis verkürzt sich der Zeitraum entsprechend. Der tatsächliche Lizenzpreis wird nach Abschluss der Beta-Phase auf Basis von Markt- und Kundenfeedback festgelegt.

3.3 Nicht-monetäre Vorteile

Neben der wirtschaftlichen Rechtfertigung ergeben sich folgende nicht-monetäre Vorteile:

- Frühzeitiger Wissensaufbau zur JTL-Cloud-API vor deren Marktreife
- Stärkung der Wettbewerbsposition von RIS als JTL-Dienstleister
- Verbesserung der Entscheidungsgrundlage für Kunden durch mobile Datenverfügbarkeit

3.4 Anwendungsfälle

Die geplante Anwendung soll sich an Händler richten, die JTL-Wawi einsetzen. Der primäre Anwendungsfall ist das Aufrufen eines Dashboards, auf dem aggregierte Geschäftskennzahlen wie Umsatz, Bestellanzahl und Lagerbestand auf einen Blick dargestellt werden sollen. Daneben soll eine Einrichtungsseite die einmalige Verbindung der App mit dem JTL-Wawi-Mandanten des Händlers ermöglichen. Ein Pane-Widget soll kontextsensitiv die ID des aktuell in der Wawi ausgewählten Kunden anzeigen. Alle drei Anwendungsfälle sind über separate Uniform Resource Locator (URL)-Pfade (`/setup`, `/erp`, `/pane`) vorgesehen und sollen von der JTL-Plattform situationsabhängig aufgerufen werden.

3.5 Lastenheft / Fachkonzept

Am Ende der Analysephase wurden die Anforderungen des Auftraggebers in einem Lastenheft festgehalten. Dieses befindet sich vollständig im Anhang [A.3](#).

4 Entwurfsphase

4.1 Zielplattform

Die Applikation wird als JTL Cloud App entwickelt, die sich nahtlos in die JTL-Cloud-Plattform integriert. Als Architektur wurde eine Monorepo-Struktur gewählt, bei der Frontend und Backend in einem gemeinsamen Quellcode-Repository verwaltet werden und geteilte Konfigurationen sowie koordinierte Builds ermöglicht werden: ein Express-Backend (TypeScript) übernimmt die Authentifizierung und API-Kommunikation, ein React-Frontend (TypeScript) stellt die Benutzeroberfläche bereit.

Für die Integration in die JTL-Plattform werden folgende JTL-spezifische Pakete eingesetzt:

- **@jtl-software/cloud-apps-core**: AppBridge, die plattformeigene Kommunikationsschicht zwischen der eingebetteten App und dem JTL-Host-System, für Inter-Process Communication (IPC)-Kommunikation mit der JTL-Wawi
- **@jtl-software/platform-ui-react**: Bibliothek vorgefertigter UI-Komponenten, die wiederverwendbare visuelle Bausteine wie Schaltflächen oder Tabellen im JTL-Design-System bereitstellt

Für die Verifikation der von der JTL-Plattform ausgestellten JSON Web Tokens (JWTs) wird die Open-Source-Bibliothek **jose** eingesetzt. Da die JTL-Plattform ihre Token mit dem Edwards-curve Digital Signature Algorithm (EdDSA)-Algorithmus signiert, einem asymmetrischen Signaturverfahren auf Basis elliptischer Kurven, war eine Bibliothek mit vollständiger EdDSA-Unterstützung erforderlich. **jose** ist kein JTL-eigenes Paket, sondern eine plattformunabhängige JWT-Bibliothek, die für diesen Anwendungsfall gewählt wurde.

Das Frontend-Framework war durch die JTL-Plattform technisch vorgegeben: Das offizielle JTL-Paket **@jtl-software/platform-ui-react** stellt ausschließlich React-spezifische UI-Komponenten bereit, und die AppBridge-Integration in **@jtl-software/cloud-apps-core** setzt React-Hooks voraus. Der Einsatz eines anderen Frameworks hätte die Kompatibilität mit diesen Plattformpaketen ausgeschlossen, weshalb eine Alternativbewertung entfällt. Darüber hinaus fügt sich **React** inhaltlich gut in das Projekt ein: Die verwendete Diagrammbibliothek **Recharts** ist eine React-native Bibliothek und ist unmittelbar auf Dashboard-Anwendungsfälle ausgelegt. **React** bietet erstklassige TypeScript-Unterstützung, was angesichts des durchgehenden TypeScript-Einsatzes im gesamten Monorepo die Konsistenz erhöht. Schließlich begünstigte die weite Verbreitung von **React** den engen Zeitrahmen von 80 h, da auf eine umfangreiche Dokumentation und erprobte Lösungsmuster zurückgegriffen werden konnte.

Für das Backend-Framework wurde eine Nutzwertanalyse zwischen den JavaScript-Frameworks **Express**, **NestJS** und **Fastify** durchgeführt, da alle drei im gleichen Ökosystem (**Node.js** / **TypeScript**) betrieben werden und somit eine vergleichbare Grundlage bilden. Die Gewichtung (Gew.) spiegelt die Bedeutung des jeweiligen Kriteriums für dieses Projekt wider; der Höchstwert 5 bedeutet unverzichtbar, 1 bedeutet vernachlässigbar. Die Bewertung (Bew.) gibt an, wie gut ein

Framework das Kriterium erfüllt: 5 = sehr gut, 1 = unzureichend. Der Nutzwert errechnet sich als Quotient aus der gewichteten Gesamtpunktzahl und der Summe aller Gewichtungen.

Tabelle 4.1: Nutzwertanalyse zur Auswahl des Backend-Frameworks

Kriterium	Gew.	Express		NestJS		Fastify	
		Bew.	Gew.	Bew.	Gew.	Bew.	Gew.
Bibliotheks-Kompatibilität	5	5	25	4	20	5	25
Einarbeitungsaufwand	5	5	25	2	10	4	20
TypeScript-Unterstützung	4	4	16	5	20	5	20
Kenntnisstand	4	5	20	2	8	2	8
Prototyp-Eignung	4	5	20	2	8	4	16
Gesamt:	22		106		66		89
Nutzwert:			4,82		3,00		4,05

Express erzielt mit einem Nutzwert von 4,82 das beste Ergebnis und wird daher für das Backend eingesetzt. Die fünf Kriterien wurden wie folgt gewichtet und bewertet:

Bibliotheks-Kompatibilität (Gew. 5): Die im Backend verwendeten Pakete `jose` und `graphql-request` sind Standard-npm-Pakete, die mit jedem Node.js-Framework funktionieren. **Express** erhält die volle Punktzahl, da es der am weitesten verbreitete Standard ist, für den alle Bibliotheken primär getestet sind; **NestJS** erhält einen Punkt Abzug, weil die zusätzliche Abstraktionsschicht gelegentlich Anpassungen erfordert.

Einarbeitungsaufwand (Gew. 5): Innerhalb des 80-Stunden-Zeitbudgets war minimaler Overhead entscheidend. **Express** erfordert kein Framework-spezifisches Architekturmuster und erlaubt den sofortigen Einstieg. **NestJS** verlangt Kenntnisse in Decorators, Dependency Injection und Modulstrukturen, was für einen Prototyp unverhältnismäßig aufwändig ist und daher mit 2 bewertet wird.

TypeScript-Unterstützung (Gew. 4): **TypeScript** ist im gesamten Monorepo durchgehend eingesetzt. **NestJS** und **Fastify** sind TypeScript-first entwickelt (Bewertung 5), während **Express** auf Community-Typings (`@types/express`) angewiesen ist (Bewertung 4).

Kenntnisstand (Gew. 4): **Express** war aus früheren Projekten bereits bekannt, was die Implementierungszeit direkt verkürzt (Bewertung 5). **NestJS** und **Fastify** waren neu und hätten zusätzliche Einarbeitungszeit erfordert (Bewertung 2).

Prototyp-Eignung (Gew. 4): **Express** erlaubt es, Endpunkte ohne Boilerplate direkt zu definieren. **NestJS** ist als Enterprise-Framework für einen Prototyp überdimensioniert und erhält daher die niedrigste Bewertung (2). **Fastify** liegt mit seinem schlanken Plugin-System dazwischen (Bewertung 4).

4.2 Architekturdesign

Das Projekt wird auf Basis einer **Client-Server-Architektur** umgesetzt. Das Backend übernimmt Datenbeschaffung, Authentifizierung und die Kommunikation mit der JTL-Cloud-API; das Frontend ist ausschließlich für die Darstellung und Benutzerinteraktion zuständig. Die Trennung von Datenzugriff und Darstellung orientiert sich an bewährten Architekturprinzipien und erleichtert die

unabhängige Weiterentwicklung beider Schichten.

Eine JTL-Cloud App ist eine vom Entwickler eigenständig gehostete Webanwendung, die über den JTL App Shell als sandboxierter `iframe` in die browserbasierte JTL-Cloud-Oberfläche eingebettet wird; ein `iframe` stellt dabei einen eingebetteten Browser-Rahmen dar, der eine separate Webanwendung innerhalb der JTL-Cloud isoliert ausführt. Der App Shell übernimmt dabei Authentifizierungstoken-Verwaltung und Kommunikation mit dem Host-System, also der JTL-Cloud-Oberfläche, die den Rahmen für den `iframe` bereitstellt; die eigentliche Anwendungslogik verbleibt vollständig auf der Seite des Entwicklers. Jeder Händler, der die App installiert, stellt einen eigenen **Tenant** dar. Die Datenisolation zwischen Mandanten ist durch das Plattformmodell sichergestellt.

Die Kommunikation zwischen App Shell und dem eingebetteten Frontend erfolgt über die JTL-Software GmbH (JTL) AppBridge (`@jtl-software/cloud-apps-core`), welche das plattformeigene IPC-Protokoll kapselt. Über die AppBridge werden Session-Tokens übermittelt sowie Methoden und Events der JTL-Wawi abgerufen.

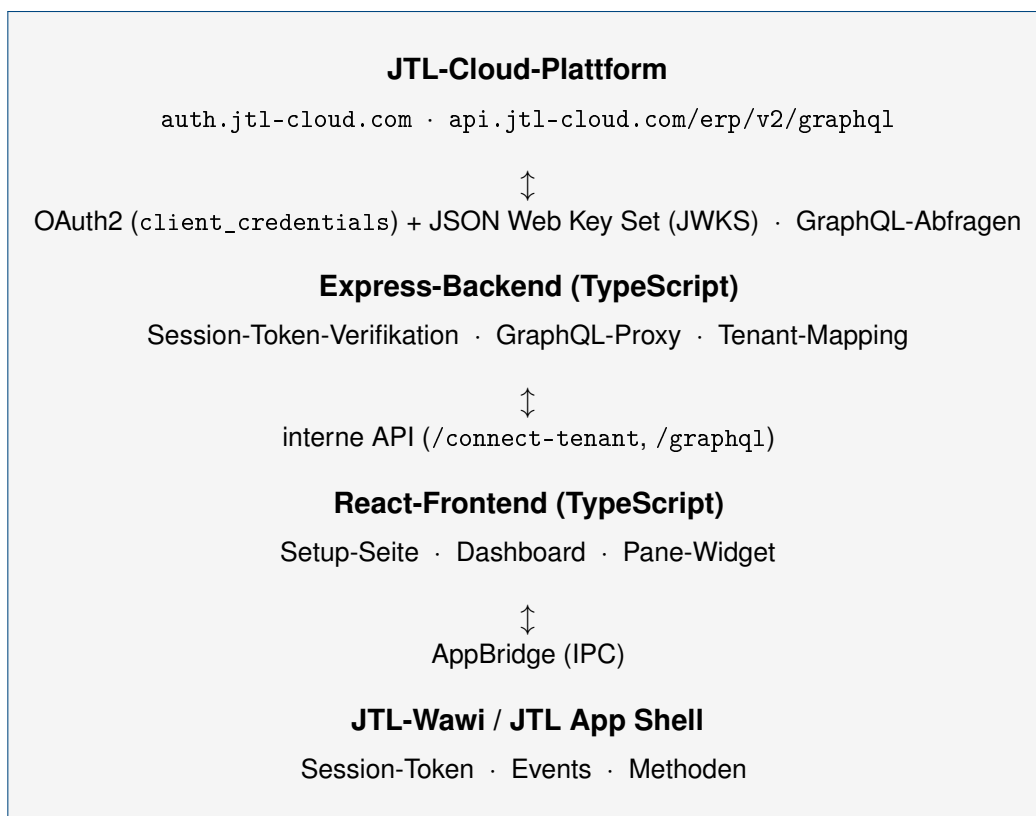


Abbildung 4.1: Systemarchitektur, Übersicht

Komponentendiagramm: Systemarchitektur
JTL Cloud BI-Tool - Abschlussprojekt Felix Bock 2026

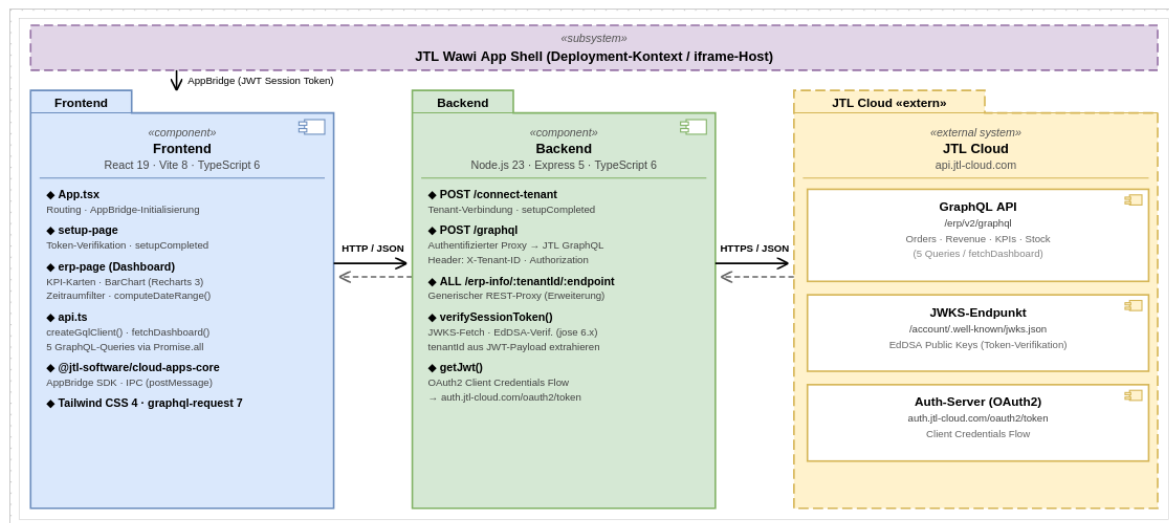


Abbildung 4.2: Komponentendiagramm der Systemarchitektur

4.3 Entwurf der Benutzeroberfläche

Um die Anwendung möglichst benutzerfreundlich zu gestalten, wurde zunächst ein Prototyp der Oberfläche als Mockup angefertigt. Dabei wurde auf eine klare, übersichtliche Darstellung der KPIs geachtet. Die Dashboard-Hauptansicht zeigt Karten für Umsatz und Auftragszahlen sowie ein Balkendiagramm für den Umsatzverlauf der letzten sieben Tage und eine Auflistung der jüngsten Bestellungen. Mockups befinden sich im Anhang [A.4](#).

4.4 Datenmodell und API-Kommunikation

4.4.1 Selbst implementierte Backend-Endpunkte

Im Rahmen des Projekts wurden im Express-Backend folgende eigene API-Endpunkte implementiert, über die das Frontend alle Anfragen abwickelt. Sie bilden eine Zwischenschicht zwischen Frontend und JTL-Cloud-API und übernehmen Authentifizierung, Token-Verifikation und Weiterleitung.

Die JTL-Cloud-API stellt neben klassischen REST-Endpunkten eine GraphQL-Schnittstelle bereit. GraphQL ist eine Abfragesprache für APIs, bei der der Client exakt vorgibt, welche Felder zurückgegeben werden sollen, anstatt fest definierte Antwortstrukturen zu empfangen. Für das Dashboard wurde diese Schnittstelle bewusst gewählt: Die fünf Anzeigebereiche des Dashboards benötigen jeweils unterschiedliche Daten aus unterschiedlichen API-Ressourcen. Mit GraphQL lassen sich diese fünf Abfragen über einen einzigen Backend-Endpunkt (/graphql) abwickeln, ohne dass für jeden Anzeigebereich ein eigener REST-Endpunkt implementiert werden müsste. Zusätzlich verhindert das feldgenaue Abfragen das Übertragen nicht benötigter Daten (Over-Fetching).

Tabelle 4.2: Selbst implementierte Backend-API-Endpunkte

Methode	Pfad	Beschreibung
POST	/connect-tenant	Verifikation des Session-Tokens und Tenant-Mapping
POST	/graphql	Authentifizierter GraphQL-Proxy zur JTL-Cloud-API
ANY	/erp-info/:tenantId/:endpoint	Generischer REST-Proxy zur JTL-Cloud-API (alle HTTP-Methoden)

Das Dashboard ruft alle Daten über den /graphql-Proxy in einer einzigen Funktion (`fetchDashboard`) ab. Die fünf dabei eingesetzten GraphQL-Operationen sowie ihre Rückgabetypen sind in Tabelle 4.3 aufgeführt.

Tabelle 4.3: GraphQL-Operationen und Rückgabetypen

Operation	Variablen	Rückgabefelder
DashboardKPIs	\$from, \$to	totalRevenue, totalOrders, totalCustomers, avgOrderValue
RevenueGrouped	\$from, \$to, \$groupBy	label, revenue, orders
TopItemsByRevenue	—	sku, name, stockInOrders, salesPriceGross, totalRevenue
QuerySalesOrders	—	salesOrderNumber, salesOrderDate, totalGrossAmount, companyName, status
LowStockItems	—	sku, name, quantityTotal, quantityAvailable, quantityLockedForShipment

Die Feldnamen entsprechen direkt den tatsächlichen Feldnamen der JTL-Cloud-GraphQL-API; sie werden ohne eigene Modellierung als TypeScript-Interfaces in `api.ts` abgebildet. Variablen für Zeitraumangaben (\$from, \$to, \$groupBy) werden von der Hilfsfunktion `computeDateRange()` berechnet und als API-Variablen übergeben.

4.4.2 JTL AppBridge-Kommunikation

Die Kommunikation zwischen der App und der JTL-Wawi erfolgt über die JTL AppBridge, einer IPC-Schnittstelle. Tabelle 4.4 zeigt die verfügbaren Methoden und Events.

Tabelle 4.4: AppBridge-Methoden und Events

Typ	Name	Beschreibung
Method	getSessionToken	Ruft das aktuelle Session-Token ab
Method	setupCompleted	Signalisiert erfolgreiche App-Einrichtung
Method	getCurrentCustomerId	Gibt die aktuelle Kunden-ID zurück
Event	CustomerChanged	Wird bei Kundenauswahl ausgelöst

4.4.3 OAuth2-Authentifizierungskonzept

Die JTL-Cloud-API erfordert eine OAuth2-Authentifizierung nach dem `client_credentials-Flow`.¹ Das Backend sendet dabei `CLIENT_ID` und `CLIENT_SECRET` als Base64-kodierte Basic-Auth-Kombination an <https://auth.jtl-cloud.com/oauth2/token> und erhält im Gegenzug ein kurzlebiges Access-Token zurück. Dieses Token wird bei jedem Aufruf der JTL-Cloud-API als `Authorization: Bearer-`Header mitgeschickt. Client-Daten werden ausschließlich als Umgebungsvariablen im Backend gehalten und sind nie im Frontend sichtbar. Der vollständige Quellcode der Funktion `getJwt()` befindet sich in Anhang A.7. Abbildung 4.3 zeigt den vollständigen Ablauf vom Session-Token-Empfang bis zur verifizierten Tenant-Zuordnung.

Sequenzdiagramm: Authentifizierung & Datenabruf

JTL Cloud BI-Tool · Abschlussprojekt Felix Bock 2026

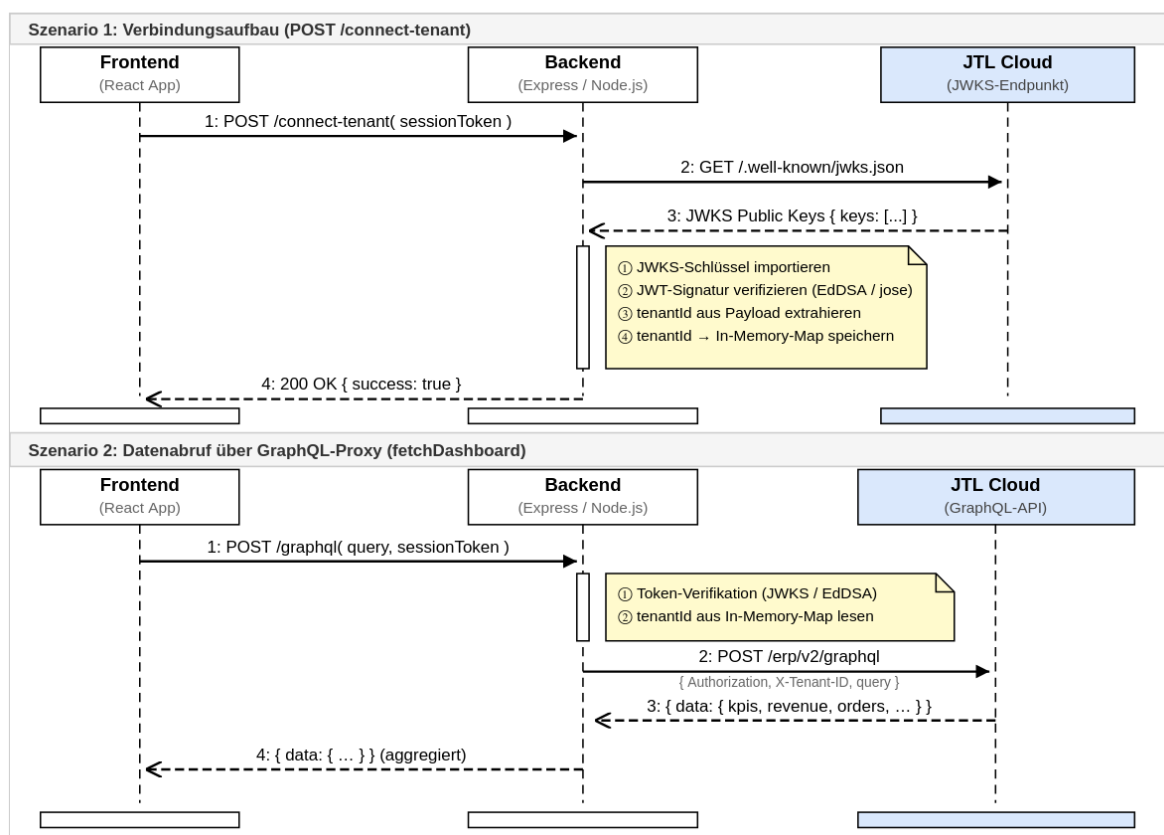


Abbildung 4.3: Sequenzdiagramm: Authentifizierungs- und Token-Verifikationsablauf

4.5 Pflichtenheft

Am Ende der Entwurfsphase wurde ein Pflichtenheft erstellt, das beschreibt, wie und womit die Anforderungen des Lastenhefts umgesetzt werden. Dieses befindet sich vollständig im Anhang A.5.

¹Vgl. JTL-Software GmbH [2026].

5 Implementierungsphase

5.1 Implementierung der Projektstruktur

Zu Beginn der Implementierungsphase wurde die Projektstruktur angelegt.

```
1 jtl-cloud-bi-app/  
2 |-- packages/  
3 |   |-- backend/                # Express-Backend  
4 |   |   |-- src/  
5 |   |   |   |-- index.ts        # Server-Einstiegspunkt  
6 |   |   |   |-- constants.ts    # Umgebungskonstanten  
7 |   |   |   |-- package.json  
8 |   |   |   |-- tsconfig.json  
9 |   |-- frontend/              # React-Frontend  
10 |       |-- src/  
11 |           |-- pages/  
12 |               |-- setup-page/ # App-Setup-Seite  
13 |               |-- erp-page/   # ERP-Integrationsseite  
14 |               |-- pane-page/  # Seitenleisten-Widget  
15 |               |-- common/     # Gemeinsame Utilities  
16 |               |-- App.tsx     # Haupt-App-Komponente  
17 |               |-- main.tsx    # React-Einstiegspunkt  
18 |               |-- package.json  
19 |-- turbo.json                 # Monorepo-Orchestrierung  
20 |-- package.json
```

Listing 5.1: Projektstruktur

5.2 Implementierung der Geschäftslogik

Da die Implementierung der OAuth2-Authentifizierung und der API-Anbindung den Kernbestandteil des Projekts darstellt, soll diese im Folgenden genauer erläutert werden.

5.2.1 Session-Token-Verifikation

Wenn der Händler die App zum ersten Mal öffnet, liefert die JTL AppBridge über `getSessionToken()` ein vom JTL-Auth-Server signiertes JWT. Das Backend verifiziert dieses Token in drei Schritten: Zunächst ruft es die öffentlichen Schlüssel vom JWKS-Endpunkt (<https://api.jtl-cloud.com/account/.well-known/jwks.json>) ab. Anschließend wird der passende Schlüssel über die Bibliothek `jose` in ein `CryptoKey`-Objekt importiert. Schließlich prüft `jwtVerify()` Signatur, Aussteller und Ablaufzeit des Tokens und extrahiert aus dem Payload die `tenantId`, die zur Identifikation des Mandanten genutzt wird. Ein ungültiges oder abgelaufenes Token führt zu einem sofortigen Abbruch mit HTTP-Statuscode 401. Die vollständige Implementierung zeigt Listing A.2 im Anhang.

5.2.2 ERP-API-Proxy

Das Backend stellt unter `/erp-info/:tenantId/:endpoint` einen generischen REST-Proxy bereit. Dieser nimmt beliebige HTTP-Methoden entgegen, ergänzt den OAuth2-Bearer-Token im Header

sowie die X-Tenant-ID zur Mandantenisolation und leitet die Anfrage an <https://api.jtl-cloud.com/erp/> (+ Endpunkt-Pfad) weiter. Der Endpunkt ist bewusst generisch gehalten, damit künftige Erweiterungen weitere JTL-Cloud-REST-Ressourcen anbinden können, ohne Änderungen am Backend vorzunehmen. Der vollständige Quellcode befindet sich in Listing A.3 im Anhang.

5.2.3 GraphQL-Datenabruf

Das zentrale Dashboard bezieht alle Anzeigedaten ausschließlich über den `/graphql`-Endpunkt des Backends, der als authentifizierter Proxy zur JTL-Cloud-GraphQL-Schnittstelle (<https://api.jtl-cloud.com/erp/v2/graphql>) fungiert. Der vollständige Datenweg vom Öffnen des Dashboards bis zur Anzeige läuft wie folgt ab: Die React-Komponente ruft beim Laden über die AppBridge das aktuelle Session-Token ab und übergibt es an `fetchDashboard()`. Diese Funktion erstellt mit `createGqlClient()` einen `graphql-request`-Client, der das Token im `X-Session-Token-Header` mitführt. Das Backend empfängt die Anfrage, prüft das Token per JWKS-Verifikation, holt sich über den `OAuth2-client_credentials`-Flow ein Access-Token und leitet die GraphQL-Abfragen mit diesem Bearer-Token an die JTL-Cloud-API weiter. Die Antwort wird an das Frontend zurückgegeben und dort in den Komponentenzustand übertragen.

Da die fünf Queries voneinander unabhängig sind, werden sie parallel über `Promise.all` ausgeführt. Bei sequenzieller Ausführung würde die Wartezeit aller fünf Anfragen addiert; durch parallele Ausführung entspricht die Gesamtladezeit der langsamsten Einzelanfrage:

```
1 const [kpisRes, revenueRes, topRes, ordersRes, stockRes] =
2   await Promise.all([
3     client.request(Q_KPIS, vars),
4     client.request(Q_REVENUE, vars),
5     client.request(Q_TOP_ITEMS),
6     client.request(Q_ORDERS),
7     client.request(Q_LOW_STOCK),
8   ]);
```

Listing 5.2: Paralleler GraphQL-Datenabruf (api.ts, Auszug)

Die Abfragen `Q_KPIS` und `Q_REVENUE` erhalten Zeitraumvariablen (`$from`, `$to`, `$groupBy`), die von `computeDateRange()` berechnet werden. Diese Hilfsfunktion nimmt einen Periodenparameter entgegen und liefert International Organization for Standardization (ISO)-Start- und Endzeitstempel sowie den passenden Gruppierungswert zurück; im Dashboard wird der Modus `'daily'` eingesetzt, der die letzten sieben Tage mit tagesweiser Gruppierung abdeckt. Die übrigen drei Queries sind zeitraumunabhängig und liefern stets aktuelle Bestands- und Bestelldaten. Das Ergebnis wird als gebündeltes `DashboardData`-Objekt an die Dashboard-Seite zurückgegeben.

Eine Herausforderung im Verlauf der Implementierung bestand darin, dass sich im Beta-Betrieb der JTL-Cloud-API einzelne Feldnamen ohne vorherige Ankündigung änderten. Dies machte es notwendig, die TypeScript-Interfaces und die GraphQL-Query-Strings anzupassen. Die Feldnamen in den Queries entsprechen daher stets dem zum Abgabezeitpunkt gültigen Schema der JTL-Cloud-API.

5.3 Implementierung der Benutzeroberfläche

Die Implementierung der Benutzeroberfläche erfolgte mit **React** 19 und der JTL Platform UI React-Komponentenbibliothek (`@jtl-software/platform-ui-react`), die vorgefertigte, im JTL-Design-System gestaltete Elemente wie Schaltflächen, Eingabefelder und Listen bereitstellt.

Eine zentrale Herausforderung bei der Frontend-Implementierung war die asynchrone Natur der AppBridge-Integration. Die AppBridge-Instanz wird einmalig beim Start der Anwendung in `main.tsx` initialisiert und steht erst nach abgeschlossenem Handshake mit der JTL-Wawi zur Verfügung. Session-Token können nur asynchron über den AppBridge-Methodenaufruf `getSessionToken()` abgerufen werden. Da alle drei Seiten der Anwendung auf diese Instanz angewiesen sind, wurde sie als Property durch die Komponentenhierarchie gereicht. Jede Seite muss daher den Token-Abruf eigenständig über Reacts `useEffect`-Hook anstoßen und den asynchronen Zustand in lokalen Zustandsvariablen verwalten.

Die Anwendung unterscheidet drei Anzeigemodi, die über den URL-Pfad bestimmt werden: `/setup` für die Einrichtungsseite, `/erp` für das Haupt-Dashboard und `/pane` für das Seitenleisten-Widget. Die App-Hauptkomponente liest den Pfad aus und rendert die passende Unterkomponente; die AppBridge-Instanz wird dabei als Property durchgereicht, damit alle Seiten auf Session-Token und JTL-Wawi-Events zugreifen können. Auf eine externe Routing-Bibliothek wurde verzichtet, da die JTL-Plattform den aufzurufenden Pfad beim Einbetten des `iframe` selbst vorgibt und kein clientseitiges Navigation-Handling erforderlich ist. Der vollständige Quellcode des Routings befindet sich in Listing [A.5](#) im Anhang.

5.3.1 Setup-Seite mit Token-Verifikation

Die Setup-Seite ist der Einstiegspunkt der App bei der Ersteinrichtung durch den Händler. Sie besteht aus einer einzelnen Schaltfläche, die beim Klick folgende Schritte auslöst: Zunächst wird das aktuelle Session-Token über den AppBridge-Aufruf `getSessionToken()` abgerufen. Dieses wird per POST-Anfrage an den `/connect-tenant`-Endpunkt des Backends gesendet, wo die Signaturprüfung und das Tenant-Mapping stattfinden. Bei erfolgreicher Verifikation signalisiert die Seite der App Shell über `setupCompleted()`, dass die Einrichtung abgeschlossen ist. Die JTL-Wawi blendet daraufhin die Setup-Seite aus und wechselt in den normalen Betriebsmodus. Die Implementierung ist in Listing [A.6](#) im Anhang dokumentiert.

5.3.2 Pane-Widget mit Event-Subscription

Das Pane-Widget demonstriert die bidirektionale Kommunikation über die AppBridge. Es wird als schmale Seitenleiste innerhalb der JTL-Wawi eingeblendet und zeigt die ID des aktuell in der Wawi ausgewählten Kunden an. Die Implementierung nutzt zwei AppBridge-Mechanismen: Über das Event `CustomerChanged` wird ein Ereignis-Listener registriert, der bei jeder Kundenauswahl in der Wawi automatisch die angezeigte Kunden-ID aktualisiert. Zusätzlich ermöglicht eine Schaltfläche die aktive Abfrage des aktuellen Kunden per AppBridge-Aufruf `getCurrentCustomerId()`. Beim Aufräumen der Komponente (React-Cleanup) wird der Event-Listener automatisch über die zurückgegebene `unsubscribe`-Funktion abgemeldet, um Speicherlecks zu vermeiden. Der Quellcode ist in Listing [A.7](#) im Anhang einsehbar.

5.3.3 ERP-Dashboard mit Kennzahlen

Die Dashboard-Seite bildet das zentrale Dashboard der Anwendung und stellt die wichtigsten Geschäftskennzahlen des jeweiligen JTL-Wawi-Mandanten dar.

Beim Laden der Komponente startet ein `useEffect`-Hook, der zunächst über den `AppBridge`-Aufruf `getSessionToken()` das aktuelle Session-Token abrufen. Während dieser asynchrone Vorgang läuft, wird ein Ladezustand angezeigt, damit die Oberfläche nicht mit leerem Inhalt erscheint. Sobald das Token vorliegt, wird es an `fetchDashboard()` übergeben, die alle fünf GraphQL-Queries parallel ausführt (vgl. Abschnitt 5.2.3). Nach Abschluss aller Anfragen werden die Ergebnisse in den React-Zustand übertragen, woraufhin die Komponente neu gerendert wird und die Daten anzeigt. Schlägt eine der Anfragen fehl, wird eine Fehlermeldung im User Interface (UI) dargestellt, ohne die Anwendung zu unterbrechen.

Die Darstellung gliedert sich in drei Bereiche: Zunächst werden zwei KPIs, Gesamtumsatz und Anzahl der Bestellungen, in kompakten Kennzahlkarten ausgegeben. Darunter folgt ein Balkendiagramm der Bibliothek `Recharts`, das den Tagesumsatz der letzten sieben Tage visualisiert. Den Abschluss bildet eine Bestellliste mit den jüngsten Aufträgen; jede Zeile enthält Firmenname, Bestellnummer, Datum, Bruttobetrag sowie einen farbcodierten Statusbadge, der den aktuellen Auftragsstatus wie „Offen“ oder „Versendet“ auf einen Blick signalisiert. Am unteren Rand wird der durchschnittliche Bestellwert als zusammenfassende Kennzahl ausgegeben.

Eine konkrete Herausforderung bei der Diagrammimplementierung bestand in der Datenaufbereitung für `Recharts`: Die JTL-Cloud-API liefert Umsatzdaten als Array von Objekten mit ISO-Zeitstempel und Betrag, während `Recharts` spezifisch benannte Felder (`name`, `value`) im Eingabearray erwartet. Die Rohdaten wurden daher in der Komponente per `.map()` in das von `Recharts` erwartete Format überführt, bevor sie an die Diagrammkomponente übergeben werden.

Für die einheitliche Formatierung der Ausgabewerte werden zwei Hilfsfunktionen eingesetzt: `formatEur()` wandelt numerische Beträge in Euro-Währungsstrings nach deutschem Gebietsschema (de-DE) um; `formatDate()` konvertiert ISO-Zeitstempel in ein lesbares Datum. Der vollständige Quellcode der Komponente befindet sich in Listing A.8 im Anhang.

Ein Screenshot des fertigen Dashboards befindet sich in Anhang A.8.

6 Abnahme- und Einführungsphase

6.1 Qualitätssicherung

Im Rahmen der Qualitätssicherung wurden Funktionstests für alle drei Anwendungsseiten (Setup, Dashboard, Pane) durchgeführt. Die Tests wurden manuell gegen die JTL-Cloud-API und über das API-Testwerkzeug **Bruno** ausgeführt; Unit-Tests einzelner Hilfsfunktionen wurden mit **Vitest** realisiert. Tabelle 6.1 zeigt eine Auswahl der durchgeführten Testfälle.

Tabelle 6.1: Testfälle Qualitätssicherung

Nr.	Testgegenstand	Erwartetes Ergebnis	Status
T1	Gültiges Session-Token an <code>/connect-tenant</code>	HTTP 200, Tenant-Mapping erstellt	Bestanden
T2	Ungültiges Session-Token an <code>/connect-tenant</code>	HTTP 401, Fehlermeldung	Bestanden
T3	GraphQL-Proxy mit gültigem Token (DashboardKPIs)	KPI-Daten aus JTL-Cloud-API zurückgegeben	Bestanden
T4	GraphQL-Proxy ohne Token-Header	HTTP 401, Anfrage abgelehnt	Bestanden
T5	Dashboard-Darstellung bei vollständigen Daten	KPI-Karten, Balkendiagramm und Bestellliste korrekt befüllt	Bestanden
T6	Zeitraumfilter <code>computeDateRange('weekly')</code>	Zurückgegebener Zeitraum deckt die letzten 4 Wochen ab	Bestanden
T7	AppBridge-Event <code>CustomerChanged</code> im Pane-Widget	Angezeigte Kunden-ID aktualisiert sich automatisch	Bestanden
T8	Setup-Ablauf Ende-zu-Ende (Token → Backend → <code>setupCompleted</code>)	App Shell wechselt nach erfolgreichem Setup in den Betriebsmodus	Bestanden

Darüber hinaus wurde, als **Fremdleistung** durch einen weiteren Entwickler des Teams der RIS, ein Code Review des erstellten Quellcodes durchgeführt. Dabei wurden Lesbarkeit, Einhaltung von Coding-Standards sowie potenzielle Sicherheitsschwachstellen bewertet. Die im Review identifizierten Anmerkungen wurden vor der finalen Abnahme eingearbeitet.

6.2 Abnahme durch den Auftraggeber

Nachdem der Prototyp fertiggestellt war, wurde dieser dem Team von RIS im Rahmen einer internen Vorstellung präsentiert. Die Abnahme sowie Bewertung der Ergebnisse erfolgte durch den zuständigen Projektbetreuer der RIS, [EINTRAGEN: Name des Betreuers], ebenfalls eine **Fremdleistung**, die nicht Bestandteil der eigenen Projektarbeit ist. Aufgrund des iterativen Vorgehens während der Entwicklung ergaben sich bei der Endabnahme keine wesentlichen Einwände.

6.3 Deployment und Einführung

Da es sich um einen Prototyp handelt, wurde die Anwendung in einer lokalen Entwicklungsumgebung bereitgestellt. Eine Produktivinstallation ist für den Fall vorgesehen, dass die JTL-Cloud-API die Beta-Phase verlässt und der Betrieb stabil läuft.

7 Dokumentation

Im Rahmen des Projekts wurde eine Kundendokumentation erstellt, die sich an Entwickler und Nutzer der Anwendung richtet. Sie umfasst zwei Teile: die technische Entwicklerdokumentation im Quellcode in Form von TypeScript-Typisierungen und JSDoc-Kommentaren sowie ein Benutzerhandbuch für die Bedienung des Dashboards. Die Kundendokumentation ist Projektbestandteil und mit 5 Stunden in der Zeitplanung berücksichtigt (entspricht 6,25 % der Gesamtprojektzeit und liegt damit innerhalb des zulässigen Rahmens von maximal 10 %). Die vollständige Entwicklerdokumentation befindet sich im Anhang [A.9](#); das Benutzerhandbuch im Anhang [A.10](#).

8 Fazit

8.1 Soll-/Ist-Vergleich

Bei einer rückblickenden Betrachtung des Projekts kann festgehalten werden, dass alle zuvor festgelegten Anforderungen gemäß dem Pflichtenheft erfüllt wurden. In Tabelle 8.1 wird die tatsächlich benötigte Zeit der geplanten Zeit gegenübergestellt.

Tabelle 8.1: Soll-/Ist-Vergleich

Projektphase	Soll	Ist	Differenz
Analysephase	10 h	10 h	0 h
Entwurfsphase	12 h	12 h	0 h
Implementierungsphase	51 h	51 h	0 h
Dokumentation	5 h	5 h	0 h
Abschluss	2 h	2 h	0 h
Gesamt	80 h	80 h	0 h

Auf Phasenebene wurden die geplanten Zeiten eingehalten. Innerhalb der Implementierungsphase ergab sich jedoch eine interne Verschiebung: Die Session-Token-Verifikation über den JWKS-Endpunkt (Iteration 4) beanspruchte zwei Stunden mehr als geplant, da dieser entgegen der verfügbaren Dokumentation eine eigene Bearer-Authentifizierung erforderte. Dieser Mehraufwand wurde durch den generischen REST-Proxy (Iteration 6) kompensiert, der aufgrund des Umstiegs auf GraphQL deutlich einfacher ausfiel als ursprünglich vorgesehen.

8.2 Lessons Learned

Die größte Herausforderung im Projektverlauf war der Beta-Status der JTL-Cloud-API. Beta-APIs befinden sich noch in der Vorabveröffentlichung, was bedeutet, dass Endpunkte, Feldnamen und Verhaltensweisen sich ohne Vorankündigung ändern können. Konkret zeigte sich dies daran, dass der JWKS-Endpunkt zur Token-Verifikation entgegen der initialen Annahme eine eigene Bearer-Authentifizierung voraussetzte. Dieser Sachverhalt war in der verfügbaren Dokumentation nicht klar beschrieben und musste durch gezielte Fehleranalyse und Rückfragen beim JTL-Support ermittelt werden. Für die Zeitplanung bedeutete dies, dass die ursprünglich für die Authentifizierung vorgesehene Zeit nicht ausreichte und im Rahmen der Implementierungsphase Anpassungen vorgenommen werden mussten.

Ein weiterer Erkenntnisgewinn betrifft den Wechsel von REST zu GraphQL als primärem Datenabrufmechanismus. Im Entwurf war ein generischer REST-Proxy vorgesehen; im Verlauf der Implementierung stellte sich heraus, dass die JTL-Cloud-API für strukturierte Datenabfragen eine deutlich leistungsfähigere GraphQL-Schnittstelle bereitstellt. Die Entscheidung, auf GraphQL umzustellen und alle fünf Dashboard-Abfragen parallel auszuführen, verbesserte sowohl die Ladezeit als auch die Typsicherheit der Datenstrukturen erheblich.

Der iterativ-inkrementelle Entwicklungsansatz bewährte sich in diesem Kontext: Durch regelmäßige Zwischenstände konnten Probleme mit der API früh erkannt und die Richtung angepasst werden, bevor größere Fehlentwicklungen entstanden.

8.3 Ausblick

Obwohl alle definierten Anforderungen realisiert werden konnten, ergeben sich für zukünftige Versionen folgende Erweiterungsmöglichkeiten:

- Erweiterung um zusätzliche KPI-Typen (z. B. Retouren, Deckungsbeiträge)
- Mandantenfähigkeit für mehrere JTL-Kunden
- Alerting-Funktion bei definierten Schwellenwerten
- Produktreife nach Wegfall des Beta-Status der JTL-Cloud-API

Aufgrund des modularen Aufbaus der Anwendung, der klaren Trennung von Backend und Frontend sowie der einheitlichen internen API, können solche Erweiterungen mit überschaubarem Aufwand vorgenommen werden.

A Anhang

A.1 Detaillierte Zeitplanung

Analysephase	10 h
Analyse der bestehenden Systemlandschaft und Problemstellung	3 h
Analyse der JTL-Cloud-API und vorhandener Schnittstellen	3 h
Machbarkeitsanalyse der technischen Umsetzung	2 h
Wirtschaftlichkeits- und Nutzenbetrachtung	2 h
Entwurfsphase	12 h
Ermittlung der technischen Anforderungen	4 h
Erstellung des technischen Konzepts (Architektur Backend/Frontend)	4 h
Entwurf der Datenstruktur und API-Kommunikation	2 h
Erstellung von UI-Mockups	2 h
Implementierungsphase	51 h
Vorbereitung Entwicklungsumgebung	7 h
Registrierung der Anwendung und API-Einrichtung	(3 h)
Einrichtung Entwicklungs-/Testumgebung	(4 h)
Backend-Entwicklung	19 h
Authentifizierung und API-Anbindung	(5 h)
Datenabfragen und -verarbeitung	(8 h)
Aufbereitung und Bereitstellung für Frontend	(6 h)
Frontend-Entwicklung	17 h
Backend-Kommunikation	(5 h)
UI-Komponenten und Kennzahlendarstellung	(8 h)
Formulare und Benutzerinteraktionen	(4 h)
Qualitätssicherung	8 h
Funktionstests	(4 h)
Code Review	(4 h)
Dokumentation	5 h
Erstellung der Entwicklerdokumentation	5 h
Abschluss	2 h
Vorstellung der Ergebnisse im Unternehmen	2 h
Gesamt	80 h

A.2 Verwendete Ressourcen

Hardware

- **Fujitsu LIFEBOOK U9310X** – Entwicklungsrechner (Laptop)

Externe Dienste

- JTL-Cloud-Testzugang – Beta-API, deren Funktionsumfang und Stabilität zum Projektzeitpunkt noch nicht final festgeschrieben waren

Software

- **Ubuntu** 24.04 Long-Term Support (LTS) – Betriebssystem
- **Visual Studio Code** – Entwicklungsumgebung
- **Node.js** 23 – Backend-Laufzeitumgebung
- **Express** 5.1 – HTTP-Server-Framework
- **React** 19 – Frontend-Framework
- **TypeScript** 6.x – Typisierte JavaScript-Erweiterung
- **Vite** 8.x – Build-Tool und Dev-Server
- **Turbo** 2.5 – Monorepo-Orchestrierung
- **Tailwind CSS** 4.x – Utility-first CSS-Framework
- **Recharts** 3.x – Diagrammbibliothek für React
- **graphql-request** 7.x – Typsicherer GraphQL-Client
- **@jtl-software/cloud-apps-core** – JTL AppBridge
- **@jtl-software/platform-ui-react** – JTL UI-Komponenten
- **jose** 6.0 – JWT-Verifikation (EdDSA)
- **Vitest** – Testframework
- **Git / GitHub** – Versionskontrolle
- **Bruno** – API-Testing
- **TeX Live** – Textsatzsystem $\text{\LaTeX}$

Personal

- Entwickler (Auszubildender) – Umsetzung des Projekts
- Mitarbeiter RIS – Fachgespräche, Code Review und Abnahme

A.3 Lastenheft

Da es sich um ein unternehmensinternes Entwicklungsprojekt der RIS handelt, wurde auf ein ausführliches externes Lastenheft verzichtet. Das nachfolgende Lastenheft dokumentiert die wesentlichen Anforderungen des Auftraggebers in vereinfachter Form.

Projektbezeichnung: JTL Wawi Cloud BI-Tool **Version:** 1.0, April 2026

Auftraggeber: RIS Web- & Software-Development GmbH & Co. KG

Auftragnehmer: Felix Bock (Auszubildender)

Ausgangssituation

Die Kunden der RIS verfügen über umfangreiche Geschäftsdaten innerhalb der JTL-Wawi (Umsätze, Bestellungen, Lagerbestände), haben jedoch keine Möglichkeit, diese mobil und visuell aufbereitet abzurufen. Die JTL-Cloud befindet sich in der Beta-Phase und stellt eine neue API-Struktur bereit, die intern bei RIS bisher nicht erprobt wurde.

Zielstellung

Ziel ist die Entwicklung eines Prototyps einer BI-Applikation als JTL Cloud App, der ausgewählte Geschäftskennzahlen aus der JTL-Cloud-API abrufen und in einem Dashboard darstellt. Gleichzeitig soll die technische Umsetzbarkeit der JTL-Cloud-API bewertet werden.

Funktionale Anforderungen

Tabelle A.1: Funktionale Anforderungen (Lastenheft)

ID	Priorität	Anforderung	Bereich
FA01	Muss	Die Anwendung authentifiziert sich über OAuth2 (client_credentials) gegenüber der JTL-Cloud-API.	Sicherheit
FA02	Muss	Die Anwendung ruft Umsatzdaten, Auftragszahlen und Lagerbestände automatisiert aus der JTL-Cloud-API ab.	Daten
FA03	Muss	Die abgerufenen Daten werden in einem grafischen Dashboard als KPI-Karten und Diagramm visualisiert.	UI
FA04	Muss	Die App lässt sich als JTL Cloud App in die JTL-Cloud-Oberfläche einbetten.	Integration
FA05	Muss	Die Session-Token-Verifikation erfolgt serverseitig über JWKS/EdDSA.	Sicherheit
FA06	Muss	Jeder Mandant sieht ausschließlich seine eigenen Daten (Tenant-Isolation).	Sicherheit
FA07	Soll	Ein Seitenleisten-Widget zeigt kontextsensitiv die Kunden-ID aus der JTL-Cloud an.	UI
FA08	Soll	Die Anwendung ist modular und erweiterbar aufgebaut.	Architektur

Nicht-funktionale Anforderungen

Tabelle A.2: Nicht-funktionale Anforderungen (Lastenheft)

ID	Bereich	Anforderung
NFA01	Sicherheit	Zugangsdaten (CLIENT_ID, CLIENT_SECRET) dürfen ausschließlich serverseitig gespeichert werden.
NFA02	Bedienbarkeit	Das Dashboard muss auf mobilen Endgeräten nutzbar sein (responsives Layout).
NFA03	Performance	Die Dashboard-Daten müssen innerhalb von 5 Sekunden nach dem Seitenaufruf angezeigt werden.
NFA04	Technologie	Die Anwendung basiert auf dem JTL-Cloud-App-Standard (AppBridge, JTL-UI-Bibliothek).
NFA05	Umfang	Es wird ein Prototyp entwickelt, kein produktionsreifes System.

Systemgrenzen

- Kein Echtzeit-Datenbetrieb für mehrere gleichzeitig aktive Mandanten (Prototyp)
- Keine Anbindung externer Systeme außerhalb der JTL-Cloud-API
- Keine Veränderung bestehender JTL-Wawi-Daten
- Kein produktionsreifer Deploymentbetrieb im Projektzeitraum

Lieferumfang

- Lauffähiger Prototyp (**Express**-Backend, **React**-Frontend, Monorepo-Struktur)
- Entwicklerdokumentation (Anhang [A.9](#))
- Benutzerhandbuch (Anhang [A.10](#))
- Projektbericht (IHK-Dokumentation)

A.4 Mockups der Benutzeroberfläche

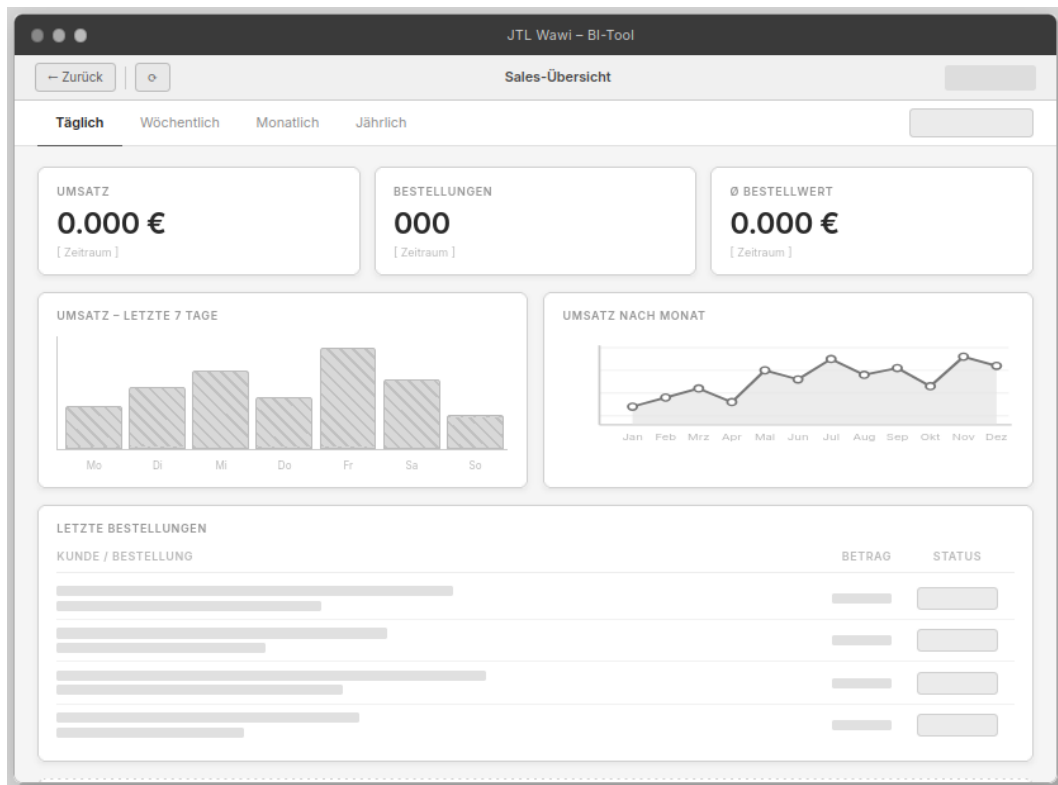


Abbildung A.1: Mockup der Dashboard-Hauptansicht

A.5 Pflichtenheft

Da es sich um ein unternehmensinternes Projekt handelt, orientiert sich das Pflichtenheft an der vereinfachten Fassung des Lastenhefts (Anhang A.3) und beschreibt, wie die dort formulierten Anforderungen technisch umgesetzt werden.

Projektbezeichnung: JTL Wawi Cloud BI-Tool **Version:** 1.0, April 2026

Auftragnehmer: Felix Bock (Auszubildender) **Auftraggeber:** RIS Web- & Software-Development GmbH & Co. KG

Systemarchitektur

Die Anwendung wird als Monorepo mit zwei Paketen umgesetzt:

- **Backend** (packages/backend): **Node.js/Express**-Server auf Port 3005. Zuständig für OAuth2-Authentifizierung, Session-Token-Verifikation und Proxying zur JTL-Cloud-API.
- **Frontend** (packages/frontend): **React-SPA**, gebildet mit **Vite**. Zuständig für Darstellung der KPI-Daten, Routing und AppBridge-Integration.

Das Backend ist der einzige Kommunikationspartner der JTL-Cloud-API. Das Frontend kommuniziert ausschließlich mit dem eigenen Backend.

Anforderungs-Umsetzungs-Matrix

Tabelle A.3: Anforderungs-Umsetzungs-Matrix (Pflichtenheft)

Anf.-ID	Komponente	Technische Umsetzung
FA01	Backend	Funktion <code>getJwt()</code> in <code>index.ts</code> : HTTP-POST an <code>auth.jtl-cloud.com/oauth2/token</code> mit Basic-Auth-Header (<code>CLIENT_ID/CLIENT_SECRET</code>) und Grant-Type <code>client_credentials</code> .
FA02	Backend	POST <code>/graphql</code> -Endpunkt: empfängt GraphQL-Abfragen vom Frontend, fügt Access-Token und Tenant-ID-Header hinzu und leitet die Anfrage an die JTL-Cloud-ERP-API weiter.
FA03	Frontend	<code>fetchDashboard()</code> in <code>api.ts</code> : sendet fünf parallele GraphQL-Abfragen (<code>Promise.all</code>). Die Ergebnisse werden als KPI-Karten und als Recharts -Balkendiagramm dargestellt.
FA04	Frontend	<code>App.tsx</code> : AppBridge-Initialisierung über <code>@jtl-software/cloud-apps-core</code> ; URL-basiertes Routing auf die Seiten <code>/erp-page</code> , <code>/setup</code> , <code>/pane-page</code> .
FA05	Backend	Funktion <code>verifySessionTokenAndExtractPayload()</code> : lädt JWKS von <code>api.jtl-cloud.com/account/.well-known/jwks.json</code> , verifiziert das JWT per EdDSA und extrahiert <code>tenantId</code> .
FA06	Backend	Die <code>tenantId</code> wird je Verbindung in einer In-Memory-Map gespeichert und bei jedem API-Aufruf als X-Tenant-ID-Header an die JTL-Cloud-API übergeben.
FA07	Frontend	<code>pane-page</code> : abonniert das <code>shell:customer-context-changed-Event</code> der AppBridge und zeigt die aktuelle Kunden-ID im Seitenleisten-Widget an.
FA08	Backend/ Frontend	Monorepo-Struktur mit Turbo : Backend- und Frontend-Paket können unabhängig weiterentwickelt werden. Neue Seiten im Frontend werden durch eine Route in <code>App.tsx</code> registriert.
NFA01	Backend	<code>CLIENT_ID</code> und <code>CLIENT_SECRET</code> werden ausschließlich über <code>.env</code> -Datei im Backend-Paket eingelesen (<code>process.env</code>); das Frontend erhält keine Zugangsdaten.

Fortsetzung auf nächster Seite

Fortsetzung von vorheriger Seite

Anf.-ID	Komponente	Technische Umsetzung
NFA02	Frontend	Tailwind CSS 4.x mit responsiven Breakpoints; das Dashboard-Grid bricht auf schmalen Bildschirmen auf eine Spalte um.
NFA03	Frontend	Fünf parallele GraphQL-Anfragen via <code>Promise.all</code> ; durch parallele Ausführung wird die Ladezeit minimiert.
NFA04	Frontend	Verwendung von <code>@jtl-software/cloud-apps-core</code> (AppBridge) und <code>@jtl-software/platform-ui-react</code> (UI-Komponenten).
NFA05	Gesamt	Kein produktionsreifes Deployment; die Anwendung läuft lokal und kann direkt im JTL Cloud App Developer Portal als Entwicklungs-App registriert werden.

Sicherheitskonzept

- **Zugangsdaten:** `CLIENT_ID` und `CLIENT_SECRET` werden ausschließlich serverseitig über Umgebungsvariablen eingelesen und nie an den Client übertragen.
- **Token-Verifikation:** Jede Frontend-Anfrage übergibt einen Session-Token (JWT), den das Backend kryptographisch gegen den öffentlichen Schlüssel des JTL-JWKS-Endpunkts verifiziert (EdDSA-Algorithmus). Erst nach erfolgreicher Verifikation wird die Anfrage an die JTL-Cloud-API weitergeleitet.
- **Tenant-Isolation:** Jedem Session-Token ist eine `tenantId` zugeordnet. Das Backend übergibt diese als `X-Tenant-ID-Header` - damit werden ausschließlich Daten des jeweiligen Mandanten zurückgegeben.
- **Cross-Origin Resource Sharing (CORS):** Das Backend erlaubt Anfragen nur vom eigenen Frontend-Ursprung.

Qualitätssicherung und Testkonzept

Die Qualitätssicherung erfolgt durch manuelle Funktionsprüfung auf zwei Ebenen:

1. **Backend-Tests:** Alle drei Endpunkte (`/connect-tenant`, `/graphql`, `/erp-info`) werden mit **Bruno** auf korrekte Responses, HTTP-Statuscodes und Fehlerbehandlung geprüft (u. a. gültige und ungültige Tokens, fehlende Parameter).
2. **Frontend-Tests:** Die Frontend-Seiten werden durch manuelle Durchführung der definierten Anwendungsfälle getestet. Dabei wird insbesondere die korrekte Darstellung der KPI-Karten, des Balkendiagramms und der Auftragsstabelle geprüft.

Vitest ist als Test-Framework im Frontend-Paket eingerichtet (`"test": "vitest run"` in `package.json`) und steht für künftige automatisierte Unit-Tests der Hilfsfunktionen (`computeDateRange`, `Formatter`) bereit. Im Rahmen des Prototyps wurde auf automatisierte Tests verzichtet.

Prototyp-Einschränkungen

Da es sich explizit um einen Prototyp handelt (vgl. NFA05), gelten folgende Einschränkungen, die für ein produktionsreifes System behoben werden müssten:

- Tenant-Mapping wird im Arbeitsspeicher (In-Memory-Map) gehalten - kein persistenter Datenbankzugriff.
- Keine Fehlerbehandlung für abgelaufene Access-Tokens außerhalb des unmittelbaren Request-Zyklus (kein automatischer Token-Refresh).
- Kein Produktions-Deployment; kein HTTPS-Terminations-Setup beschrieben.

A.6 Iterationsplan

Iter.	Tätigkeit	Soll(h)	Ist(h)	Diff.
<i>Vorbereitung / Entwurfsphase</i>				
1	Einrichtung Monorepo-Struktur, TypeScript- und Vite-Konfiguration	4	4	0
2	Registrierung der App im JTL Developer Portal, API-Einrichtung	3	3	0
<i>Implementierungsphase</i>				
3	Implementierung OAuth2-Authentifizierung (Backend, getJwt)	5	5	0
4	Implementierung Session-Token-Verifikation mit JWKS	5	7	+2
5	Implementierung GraphQL-Proxy-Route (/graphql)	4	4	0
6	Implementierung generischer REST-Proxy (/erp-info)	3	1	-2
7	Implementierung React-App-Routing und AppBridge-Integration	5	5	0
8	Implementierung Setup-Seite mit Token-Flow	4	4	0
9	Implementierung ERP-Dashboard: GraphQL-Datenabruf, KPI-Karten, Chart	8	8	0
10	Implementierung Pane-Widget mit Event-Subscription	4	4	0
11	Integration JTL Platform UI Komponenten, Styling	5	5	0
12	Funktionstests, Code Review, Fehlerbehebung	8	8	0
<i>Dokumentation und Abschluss</i>				
13	Entwicklerdokumentation (JSDoc, Typisierungen)	5	5	0
14	Vorstellung, Abnahme, Abschluss	2	2	0
Gesamt		65	65	0

A.7 Quellcode-Auszüge

Nachfolgend sind die zentralen Quellcode-Auszüge des Projekts dokumentiert.

A.7.1 Backend: OAuth2-Token-Anfrage

```
1 export async function getJwt(): Promise<string> {
2   const authString = Buffer.from(
```

```
3   `${process.env.CLIENT_ID}:${process.env.CLIENT_SECRET}`  
4   ).toString('base64');  
5  
6   const response = await fetch(  
7     'https://auth.jtl-cloud.com/oauth2/token',  
8     {  
9       method: 'POST',  
10      headers: {  
11        'Content-Type': 'application/x-www-form-urlencoded',  
12        'Authorization': 'Basic ${authString}',  
13      },  
14      body: new URLSearchParams({ grant_type: 'client_credentials' }),  
15    }  
16  );  
17  const data = await response.json();  
18  if (response.ok) return data.access_token;  
19  throw new Error('JWT-Fehler: ${data.error}');  
20 }
```

Listing A.1: OAuth2-Token-Anfrage mit Basic Auth (index.ts)

A.7.2 Backend: Session-Token-Verifikation

```
1 import { importJWK, jwtVerify } from 'jose';  
2  
3 const JWKS_URL =  
4   'https://api.jtl-cloud.com/account/.well-known/jwks.json';  
5  
6 async function verifySessionToken(  
7   sessionToken: string  
8 ): Promise<SessionTokenPayload> {  
9   const jwt = await getJwt();  
10  const response = await fetch(JWKS_URL, {  
11    headers: { Authorization: 'Bearer ${jwt}' },  
12  });  
13  const jwks = await response.json();  
14  const publicKey = await importJWK(jwks.keys[0], 'EdDSA');  
15  const { payload } = await jwtVerify(sessionToken, publicKey);  
16  return payload as SessionTokenPayload;  
17 }
```

Listing A.2: Session-Token-Verifikation mit JWKS (index.ts)

A.7.3 Backend: ERP-API-Proxy-Route

```
1 app.all('/erp-info/:tenantId/:endpoint',  
2   async (req: Request, res: Response) => {  
3     const { tenantId, endpoint } = req.params;  
4     const jwt = await getJwt();  
5     const options: RequestInit = {  
6       method: req.method,  
7       headers: {  
8         'X-Tenant-ID': tenantId,
```

```
9      'Authorization': 'Bearer ${jwt}',
10      'Content-Type': 'application/json',
11    },
12  };
13  if (['POST', 'PUT', 'PATCH'].includes(req.method))
14    options.body = JSON.stringify(req.body);
15  const erpResponse = await fetch(
16    'https://api.jtl-cloud.com/erp/${endpoint}', options
17  );
18  if (erpResponse.ok) res.json(await erpResponse.json());
19  else res.status(erpResponse.status).send(await erpResponse.text());
20  }
21  );
```

Listing A.3: Generischer REST-Proxy-Endpunkt (index.ts)

A.7.4 Backend: Tenant-Mapping

```
1  const myMappingDatabase = new Map<string, string>();
2
3  app.post('/connect-tenant', async (req, res) => {
4    const { sessionToken } = req.body;
5    const payload = await verifySessionToken(sessionToken);
6    const tenantId = new Date().getTime().toString();
7    myMappingDatabase.set(tenantId, payload.tenantId);
8    res.send('Tenant ${tenantId} verbunden');
9  });
```

Listing A.4: Tenant-Mapping Endpunkt (index.ts)

A.7.5 Frontend: App-Routing

```
1  const App: React.FC<{ appBridge: AppBridge }> = ({ appBridge }) => {
2    const mode = location.pathname.substring(1) as AppMode;
3    switch (mode) {
4      case 'setup': return <SetupPage appBridge={appBridge} />;
5      case 'erp':   return <ErpPage   appBridge={appBridge} />;
6      case 'pane':  return <PanePage  appBridge={appBridge} />;
7      default:      return <div>Unknown mode</div>;
8    }
9  };
```

Listing A.5: App-Komponente mit URL-basiertem Routing (App.tsx)

A.7.6 Frontend: Setup-Seite

```
1  const SetupPage: React.FC<ISetupPageProps> = ({ appBridge }) => {
2    const [isLoading, setIsLoading] = useState(false);
3
4    const handleSetupCompleted = useCallback(async () => {
5      setIsLoading(true);
6      const sessionToken = await appBridge.method.call('getSessionToken');
7      const response = await fetch('/connect-tenant', {
```

```
8     method: 'POST',
9     headers: { 'Content-Type': 'application/json' },
10    body: JSON.stringify({ sessionToken }),
11  });
12  if (response.ok)
13    await appBridge.method.call('setupCompleted');
14    setIsLoading(false);
15  }, [appBridge]);
16
17  return (
18    <div>
19      <h1>Setup</h1>
20      <Button onClick={handleSetupCompleted} label="App verbinden" />
21    </div>
22  );
23  };
```

Listing A.6: Setup-Seite mit AppBridge-Token-Flow (SetupPage.tsx)

A.7.7 Frontend: Pane-Widget

```
1  const PanePage: React.FC<IPanePageProps> = ({ appBridge }) => {
2    const [customer, setCustomer] = useState<string>();
3
4    useEffect(() => {
5      const unsubscribe = appBridge.event.subscribe(
6        'CustomerChanged',
7        (data: { customerId: string }) => {
8          setCustomer(data.customerId);
9          return Promise.resolve();
10        }
11      );
12      return () => unsubscribe();
13    }, [appBridge]);
14
15    return (
16      <Stack spacing="4" direction="column">
17        <Text align="center" type="h2">Pane App</Text>
18        <Input disabled value={customer} />
19        <Button
20          variant="outline"
21          onClick={() => appBridge.method.call('getCurrentCustomerId')
22            .then(id => setCustomer(id as string))}
23          label="Aktuellen Kunden abrufen"
24        />
25      </Stack>
26    );
27  };
```

Listing A.7: Pane-Widget mit Event-Subscription (PanePage.tsx)

A.7.8 Frontend: ERP-Dashboard-Komponente

```
1 const ErpPage: React.FC<IErpPageProps> = ({ appBridge }) => {
2   const [data, setData] = useState<DashboardData | null>(null);
3
4   const load = useCallback(async () => {
5     const token = await appBridge.method.call<string>('getSessionToken');
6     const result = await fetchDashboard(token, computeDateRange('daily'));
7     setData(result);
8   }, [appBridge]);
9
10  useEffect(() => { load(); }, [load]);
11
12  return (
13    <div className="flex flex-col gap-4 p-4 bg-gray-50">
14      <div className="flex gap-3">
15        <KpiCard label="Umsatz" value={formatEur(data.kpis.totalRevenue)} />
16        <KpiCard label="Bestellungen" value={String(data.kpis.totalOrders)} />
17      </div>
18      <BarChart data={data.revenuePoints.slice(-7)}>
19        <Bar dataKey="revenue" fill="#1a56db" />
20      </BarChart>
21      {data.recentOrders.slice(0, 8).map(order => (
22        <div key={order.salesOrderNumber}>
23          <p>{order.companyName}</p>
24          <p>{formatEur(order.totalGrossAmount)}</p>
25          <StatusBadge status={order.status} />
26        </div>
27      ))}
28      <p>Avg. Bestellwert: {formatEur(data.kpis.avgOrderValue)}</p>
29    </div>
30  );
31 };
```

Listing A.8: ERP-Dashboard: Datenabruf und Darstellung (ErpPage.tsx, Auszug)

A.7.9 Frontend: React-Einstiegspunkt

```
1 import { createAppBridge } from '@jtl-software/cloud-apps-core';
2 import { StrictMode } from 'react';
3 import { createRoot } from 'react-dom/client';
4 import App from './App.tsx';
5
6 const appBridge = createAppBridge();
7
8 createRoot(document.getElementById('root')!).render(
9   <StrictMode>
10     <App appBridge={appBridge} />
11   </StrictMode>
12 );
```

Listing A.9: main.tsx mit AppBridge-Initialisierung

A.8 Screenshot der Anwendung

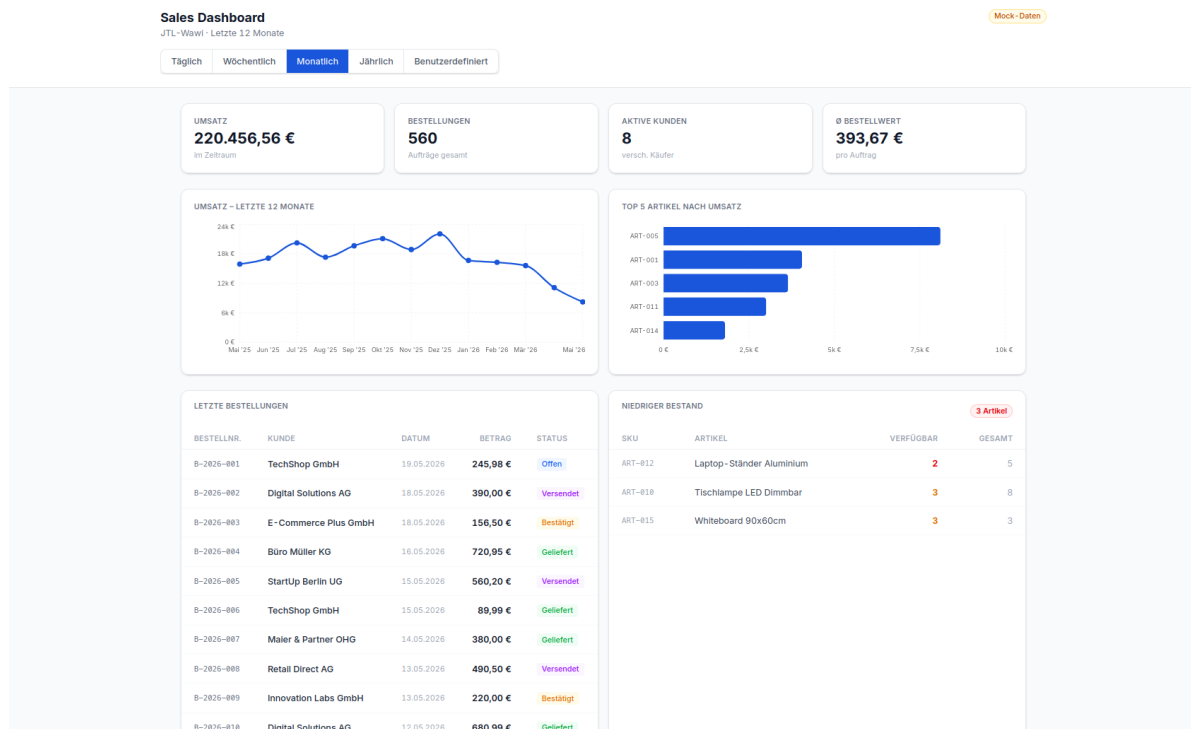


Abbildung A.2: Screenshot des fertigen ERP-Dashboards

A.9 Entwicklerdokumentation

Diese Entwicklerdokumentation richtet sich an Entwickler der RIS, die das Projekt weiterentwickeln oder in eine eigene Infrastruktur integrieren möchten. Sie beschreibt Einrichtung, API-Referenz und zentrale Datenstrukturen.

Voraussetzungen und Einrichtung

1. **Node.js** ≥ 23 und **npm** ≥ 10 müssen installiert sein.
2. Repository klonen und Abhängigkeiten installieren:

```
1 git clone <repository-url>
2 cd felix-projekt
3 npm install
```

Listing A.10: Projekt einrichten

3. Im Verzeichnis `packages/backend/` eine `.env`-Datei mit den Zugangsdaten aus dem JTL Developer Portal anlegen:

```
1 CLIENT_ID=<app-client-id-aus-dem-jtl-developer-portal>
2 CLIENT_SECRET=<app-client-secret-aus-dem-jtl-developer-portal>
```

Listing A.11: Umgebungsvariablen (`packages/backend/.env`)

4. Optionale Frontend-Variable: Zeigt das Frontend auf ein anderes Backend, kann in `packages/frontend/.env` gesetzt werden:

```
1 VITE_API_URL=http://localhost:3005 # Standard, kann weggelassen werden
```

Listing A.12: Optionale Frontend-Variable

5. Entwicklungsserver starten (startet Backend und Frontend parallel via **Turbo**):

```
1 npm run dev
```

Listing A.13: Entwicklungsserver starten

Das Backend ist anschließend unter `http://localhost:3005` erreichbar, das Frontend unter `http://localhost:5173`.

Backend-API-Referenz

Alle Endpunkte sind unter `http://localhost:3005` erreichbar. Der `session-token`-Header muss bei allen schreibenden Aufrufen übergeben werden.

POST `/connect-tenant` Verknüpft einen Session-Token mit einer Mandanten-Kennung.

```
1 POST /connect-tenant
2 Content-Type: application/json
3
4 {
5   "sessionToken": "<jwt-vom-appbridge-event>"
6 }
```

Listing A.14: POST `/connect-tenant` - Beispiel-Request

```
1 HTTP 200 OK
2 {
3   "tenantId": "tenant-1746351234567"
4 }
```

Listing A.15: POST `/connect-tenant` - Erfolgs-Response

Fehlerfall: Ungültiger Token liefert HTTP 401 mit `{error: "...}"`.

POST `/graphql` Proxied eine GraphQL-Abfrage an die JTL-Cloud-ERP-API. Das Backend fügt `Access-Token` und `X-Tenant-ID-Header` automatisch hinzu.

```
1 POST /graphql
2 Content-Type: application/json
3 session-token: <jwt-vom-appbridge-event>
4
5 {
6   "query": "{ orders(first: 5) { nodes { orderId status } } }",
7   "variables": {}
8 }
```

Listing A.16: POST `/graphql` - Beispiel-Request

Die Response entspricht direkt der GraphQL-Response der JTL-Cloud-API (`{"data": {...}, errors: [...]}`).

ALL `/erp-info/:tenantId/:endpoint` Generischer REST-Proxy für künftige Erweiterungen. Leitet beliebige HTTP-Methoden an `https://api.jtl-cloud.com/erp/:endpoint` weiter. Wird vom aktuellen Dashboard-Frontend nicht genutzt.

Frontend-Funktionsreferenz

Alle nachfolgenden Funktionen befinden sich in `packages/frontend/src/common/api.ts`.

```
1  /**
2   * Ruft alle Dashboard-KPIs eines Mandanten parallel ab.
3   * Fuehrt fuenf GraphQL-Abfragen gleichzeitig aus (Promise.all).
4   *
5   * @param sessionToken - Aktueller Session-Token aus der AppBridge
6   * @param range - Zeitraum fuer Datenabruf ('daily'/'weekly'/'
7   *               'monthly'/'yearly'/'custom')
8   * @returns Promise<DashboardKPIs> mit allen KPI-Werten
9   * @throws Error bei Netzwerkfehler oder ungueltigem Token
10  *
11  * @example
12  * const data = await fetchDashboard(token, 'monthly');
13  * console.log(data.totalRevenue); // z. B. 48320.50
14  */
15  async function fetchDashboard(
16    sessionToken: string,
17    range: DateRangeMode
18  ): Promise<DashboardKPIs>
```

Listing A.17: JSDoc-Dokumentation: `fetchDashboard()`

```
1  /**
2   * Berechnet Start- und Enddatum fuer einen benannten Zeitraum.
3   *
4   * @param mode - 'daily' | 'weekly' | 'monthly' | 'yearly' | 'custom'
5   * @param customFrom - Nur bei mode='custom': ISO-Datumsstring Start
6   * @param customTo - Nur bei mode='custom': ISO-Datumsstring Ende
7   * @returns DateRange mit { from: string, to: string } (ISO-8601)
8   *
9   * @example
10  * computeDateRange('monthly');
11  * // { from: '2026-04-01', to: '2026-04-30' }
12  *
13  * computeDateRange('custom', '2026-01-01', '2026-03-31');
14  * // { from: '2026-01-01', to: '2026-03-31' }
15  */
16  function computeDateRange(
17    mode: DateRangeMode,
18    customFrom?: string,
19    customTo?: string
20  ): DateRange
```

Listing A.18: JSDoc-Dokumentation: computeDateRange()

```
1  /**
2   * Formatiert einen Eurobetrag als deutschsprachigen Geldwert.
3   * @param value - Betrag in Euro (Zahl)
4   * @returns Formatierter String, z. B. "1.234,56 EUR"
5   */
6  function formatEur(value: number): string
7
8  /**
9   * Formatiert ein ISO-Datum als deutsches Kurzformat.
10  * @param iso - ISO-8601-Datumsstring
11  * @returns Formatierter String, z. B. "15.04.2026"
12  */
13 function formatDate(iso: string): string
```

Listing A.19: JSDoc-Dokumentation: Hilfsfunktionen

TypeScript-Schnittstellen

```
1  /** Alle Dashboard-Kennzahlen eines Mandanten */
2  interface DashboardKPIs {
3      totalRevenue: number; // Gesamtumsatz im Zeitraum
4      orderCount: number; // Anzahl Auftraege
5      avgOrderValue: number; // Durchschnittlicher Auftragswert
6      revenueByDay: RevenuePoint[]; // Umsatz je Tag (fuer Balkendiagramm)
7      topProducts: TopProduct[]; // Top-Produkte nach Umsatz
8      recentOrders: Order[]; // Letzte Auftraege
9      lowStockItems: LowStockItem[]; // Artikel unter Mindestbestand
10 }
11
12 /** Einzelner Datenpunkt fuer das Umsatzdiagramm */
13 interface RevenuePoint {
14     date: string; // ISO-Datum
15     revenue: number; // Tagesumsatz in Euro
16 }
17
18 /** Auftragsstatus-Mapping */
19 type OrderStatus =
20     'open' | 'confirmed' | 'shipped' | 'delivered' | 'cancelled';
21
22 /** Einzelner Auftrag */
23 interface Order {
24     orderId: string;
25     status: OrderStatus;
26     totalValue: number;
27     orderDate: string;
28 }
29
30 /** Artikel mit niedrigem Lagerbestand */
31 interface LowStockItem {
```

```
32   productId:    string;  
33   productName:  string;  
34   stockQuantity: number;  
35   minStock:     number;  
36 }
```

Listing A.20: TypeScript-Interfaces (api.ts)

Test-Setup

Vitest ist als Test-Framework konfiguriert und kann über das npm-Script gestartet werden, sobald Testdateien angelegt werden:

```
1  cd packages/frontend  
2  npx vitest run
```

Listing A.21: Test-Script starten (packages/frontend)

Für künftige Tests empfehlen sich Unit-Tests der Hilfsfunktionen `computeDateRange`, `formatEur` und `formatDate` in `packages/frontend/src/__tests__/`.

A.10 Benutzerhandbuch

Dieses Benutzerhandbuch richtet sich an JTL-Händler, die das BI-Dashboard als JTL Cloud App verwenden. Es beschreibt, wie die Anwendung gestartet, eingerichtet und im Alltag genutzt wird.

Über die Anwendung

Das JTL Wawi Cloud BI-Tool ist eine Cloud App, die direkt in der JTL-Cloud-Oberfläche eingebettet läuft. Es zeigt ausgewählte Geschäftskennzahlen des verbundenen JTL-Wawi-Mandanten in einem übersichtlichen Dashboard: Umsatz, Auftragsanzahl, durchschnittlicher Auftragswert, Top-Produkte, aktuelle Aufträge und Artikel mit niedrigem Lagerbestand.

Voraussetzungen

- Aktiver JTL-Cloud-Zugang mit installierter App
- JTL-Wawi-Mandant mit vorhandenen Auftragsdaten
- Unterstützter Browser: aktuelle Version von Chrome, Firefox oder Edge

Ersteinrichtung

Beim ersten Aufruf der App öffnet sich automatisch die **Setup-Seite**:

1. Klick auf „*Verbindung herstellen*“: Die App überträgt den Session-Token aus der JTL-Cloud automatisch an das Backend und legt eine Mandantenverbindung an.
2. Nach erfolgreicher Verifikation erscheint die Bestätigung „*Verbindung hergestellt*“ und die **ID des Mandanten**.
3. Über den Link „*Zum Dashboard*“ wechselt die App zur Dashboard-Ansicht. Bei jedem weiteren Aufruf wird die Setup-Seite automatisch übersprungen, sofern die Session noch gültig ist.

Das Dashboard

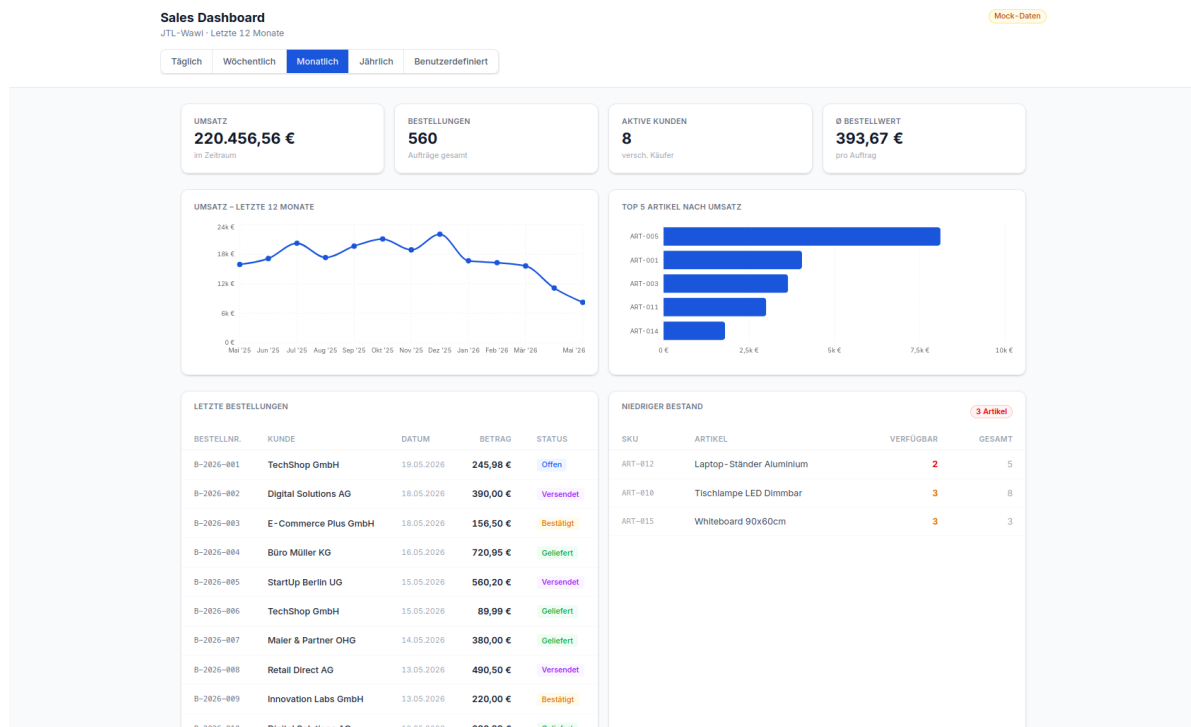


Abbildung A.3: Dashboard-Ansicht (Screenshot)

Das Dashboard besteht aus vier Bereichen:

KPI-Karten (oben) Vier Kennzahlenkarten zeigen auf einen Blick:

- **Gesamtumsatz** im aktuellen Monat in Euro
- **Anzahl Aufträge** im aktuellen Monat
- **Durchschnittlicher Auftragswert** in Euro
- **Artikel unter Mindestbestand** (Anzahl)

Umsatzdiagramm (Mitte) Ein Balkendiagramm stellt den täglichen Umsatzverlauf des aktuellen Monats dar. Die horizontale Achse zeigt das Datum, die vertikale Achse den Tagesumsatz in Euro.

Letzte Aufträge (unten links) Eine Tabelle listet die zuletzt eingegangenen Aufträge mit Auftrags-ID, Datum, Gesamtwert und aktuellem Status. Der Status wird farbig hervorgehoben:

Tabelle A.4: Auftragsstatus im Dashboard

Status	Bedeutung
Offen	Auftrag eingegangen, noch nicht bestätigt
Bestätigt	Auftrag angenommen, in Bearbeitung
Versendet	Paket übergeben an Versanddienstleister
Geliefert	Zustellung bestätigt
Storniert	Auftrag wurde storniert

Niedrige Lagerbestände (unten rechts) Eine Liste zeigt alle Artikel, deren aktueller Lagerbestand den definierten Mindestbestand unterschreitet, mit Produktname und verbleibendem Bestand.

Seitenleisten-Widget (Pane)

Das Seitenleisten-Widget erscheint innerhalb der JTL-Cloud-Oberfläche in der rechten Seitenleiste. Es zeigt kontextsensitiv die Kunden-ID des gerade in der JTL-Cloud geöffneten Kundendatensatzes an. Das Widget aktualisiert sich automatisch, wenn in der JTL-Cloud zu einem anderen Kunden gewechselt wird.

Fehlerbehebung

Tabelle A.5: Häufige Probleme und Lösungen

Problem	Lösung
Dashboards zeigt keine Daten	Seite neu laden; bei erneutem Ausbleiben die Setup-Seite erneut aufrufen.
Fehlermeldung „Token ungültig“	JTL-Cloud-Sitzung abgelaufen - erneut in der JTL-Cloud anmelden.
Kein Umsatz angezeigt	Prüfen ob im gewählten Zeitraum Aufträge im JTL-Wawi-Mandanten vorliegen.
App lädt nicht	Sicherstellen, dass das Backend unter <code>http://localhost:3005</code> erreichbar ist.

A.11 Eigenständigkeitserklärung

Ich versichere hiermit, dass ich die vorliegende Projektarbeit selbstständig und ohne unzulässige fremde Hilfe angefertigt habe. Alle Stellen, die ich wörtlich oder sinngemäß aus veröffentlichten oder nicht veröffentlichten Quellen entnommen habe, sind als solche kenntlich gemacht.

Tätigkeiten, die nicht von mir selbst durchgeführt wurden, sind in der Dokumentation ausdrücklich als Fremdleistung gekennzeichnet (insbesondere Code Review und Abnahme durch Mitarbeitende der RIS).

Die Dokumentation wurde gemäß den Vorgaben der Industrie- und Handelskammer (IHK) Regensburg erstellt und enthält keine unternehmensinternen Informationen, die nicht zur Veröffent-

lichung freigegeben sind.

Ort, Datum

Unterschrift Felix Bock